

《神经网络与深度学习》

总复习



周晓飞
自动化学院

《神经网络与深度学习》

绪论



周晓飞
自动化学院

人工智能 artificial intelligence, AI

- ▶ 人工智能（artificial intelligence, AI）就是让机器具有人类的智能。
 - ▶ “计算机控制” + “智能行为”
- ▶ 1950年，**阿兰.图灵**提出“**图灵测试**”
- ▶ 人工智能这个学科的诞生有着明确的标志性事件，就是1956年的达特茅斯（Dartmouth）会议。在这次会议上，“人工智能”被提出并作为本研究领域的名称。

人工智能就是要让机器的行为看起来就像是人所表现出的智能行为一样。

John McCarthy (1927-2011)

人工智能的研究领域

▶ 让机器具有人类的智能（感知 学习 认知）

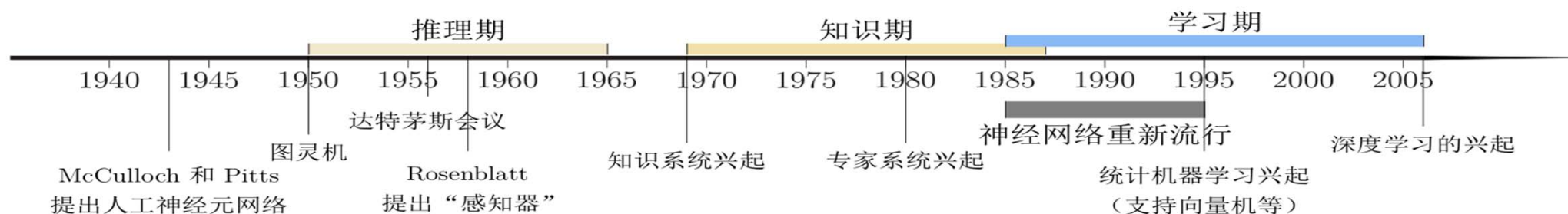
- ▶ 机器感知（计算机视觉、语音信息处理）
- ▶ 学习（模式识别、机器学习、强化学习）
- ▶ 语言（自然语言处理）
- ▶ 记忆（知识表示）
- ▶ 决策（规划、数据挖掘）

人工智能发展历史

推理期：逻辑 + 规则

知识期：专家系统 (**Expert System**)

学习期：机器学习 (**Machine Learning**)



机器学习

- ▶ 当我们用机器学习来解决一些模式识别任务时，一般的流程包含以下几个步骤：



特征工程 (Feature Engineering)

- ▶ 浅层学习 (Shallow Learning)：不涉及特征学习，其特征主要靠人工经验或特征转换方法来抽取。

三、表示学习

Bengio, Yoshua, Aaron Courville, and Pascal Vincent.
"Representation learning: A review and new perspectives." IEEE
transactions on pattern analysis and machine intelligence 35.8 (2013):
1798-1828.

- ▶ 数据表示是机器学习的核心问题。
 - ▶ 特征工程：需要借助人脑智能
- ▶ 表示学习（representation learning）
 - ▶ 如何自动从数据中学习出有效的特征，来提高机器学习模型的性能
- ▶ 难点
 - ▶ 没有明确的目标

语义鸿沟： Semantic Gap

- ▶ 表示学习的关键；人工智能的挑战之一
- ▶ 即输入数据的底层特征和高层语义信息之间的不一致性和差异性
 - ▶ 人们对文本、图像的理解无法从字符串或者图像的底层特征直接获得



床 前 明 月 光 ，
疑 是 地 上 霜 。
举 头 望 明 月 ，
低 头 思 故 乡 。

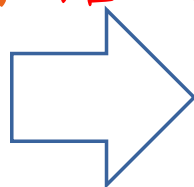
什么是好的数据表示？

- ▶ “好的表示”是一个非常主观的概念，没有一个明确的标准。
- ▶ 但一般而言，一个好的表示具有以下几个优点：
 - ▶ 应该具有很强的表示能力
 - ▶ 应该使后续的学习任务变得简单，包含更高层的语义信息
 - ▶ 应该具有一般性，是任务或领域独立的，可以迁移到其他任务上
- ▶ 机器学习任务中，经常采用两种方式来表示特征：局部表示(Local Representation) & 分布式表示 (Distributed Representation)

语义表示

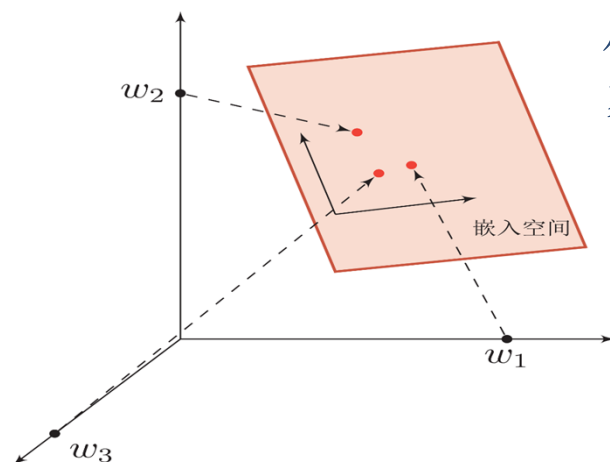
► 如何在计算机中表示语义?

局部表示



分布式表示

One-Hot向量
局部表示



嵌入embedding: 通常指将一个度量空间中的对象映射到另一个低维度的度量空间中, 并尽可能的保持不同对象之间的拓补关系

嵌入: 压缩、
低维、稠密向量

表示形式

▶ 符号表示

▶ 离散表示、局部表示

▶ One-Hot向量

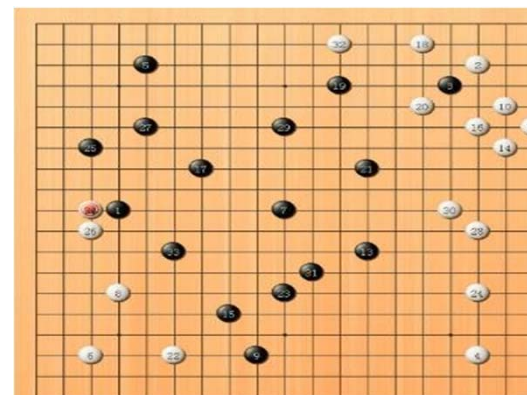
	离散表示
A	[1 0 0 0]
B	[0 1 0 0]
C	[0 0 1 0]
D	[0 0 0 1]

▶ 分布式(distributed)表示

▶ 压缩、低维、稠密向量

▶ 用 $O(N)$ 个参数表示 $O(2^k)$ 区间

▶ k 为非0参数, $k < N$



分布式表示

深度学习 Deep Learning

- ▶ **端到端学习**（**端到端训练, End-to-end Learning**）：在学习过程中不进行分模块或者分阶段训练，直接优化任务的总体目标。中间过程无需人为干预，目前大部分采用神经网络的深度学习可以看作是一种端到端的学习。
- ▶ 端到端学习亦要解决贡献度分配问题。目前大部分采用的神经网络模型的深度学习也可以看做一种端到端的学习。

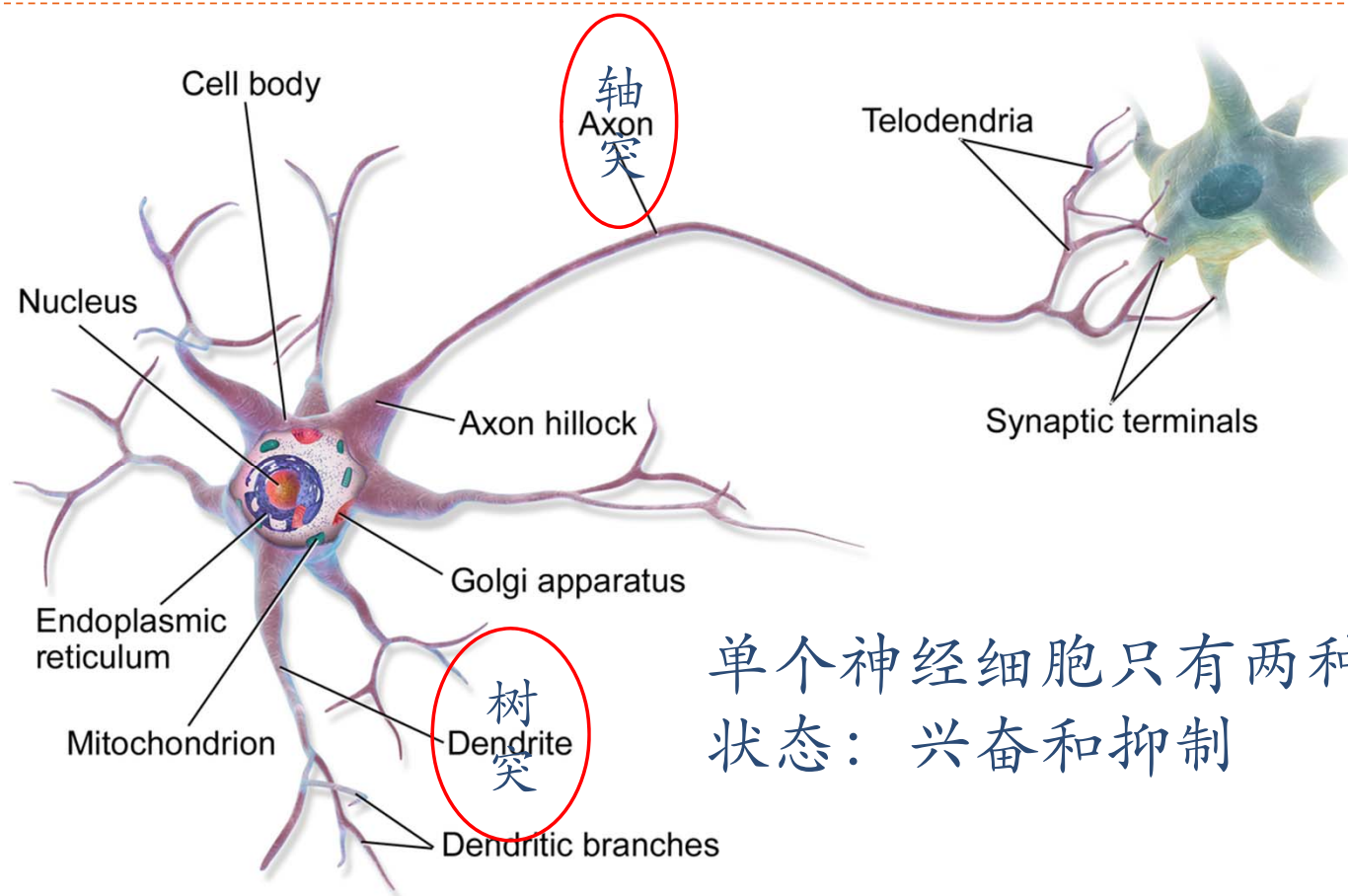
神经网络 (Neural Network)

- ▶ **人工神经网络 (Artificial Neural Network, ANN)** 是指一系列受生物学和神经科学启发的数学模型。其主要通过对人脑神经元网络进行抽象，构建人工神经单元，并按照一定的拓补结构来建立人工神经元之间的连接，来模拟生物神经网络。
- ▶ 人工神经网络亦常被简称为**神经网络 (Neural Network, NN)** 或**神经模型 (Neural Model)**

人工神经网络

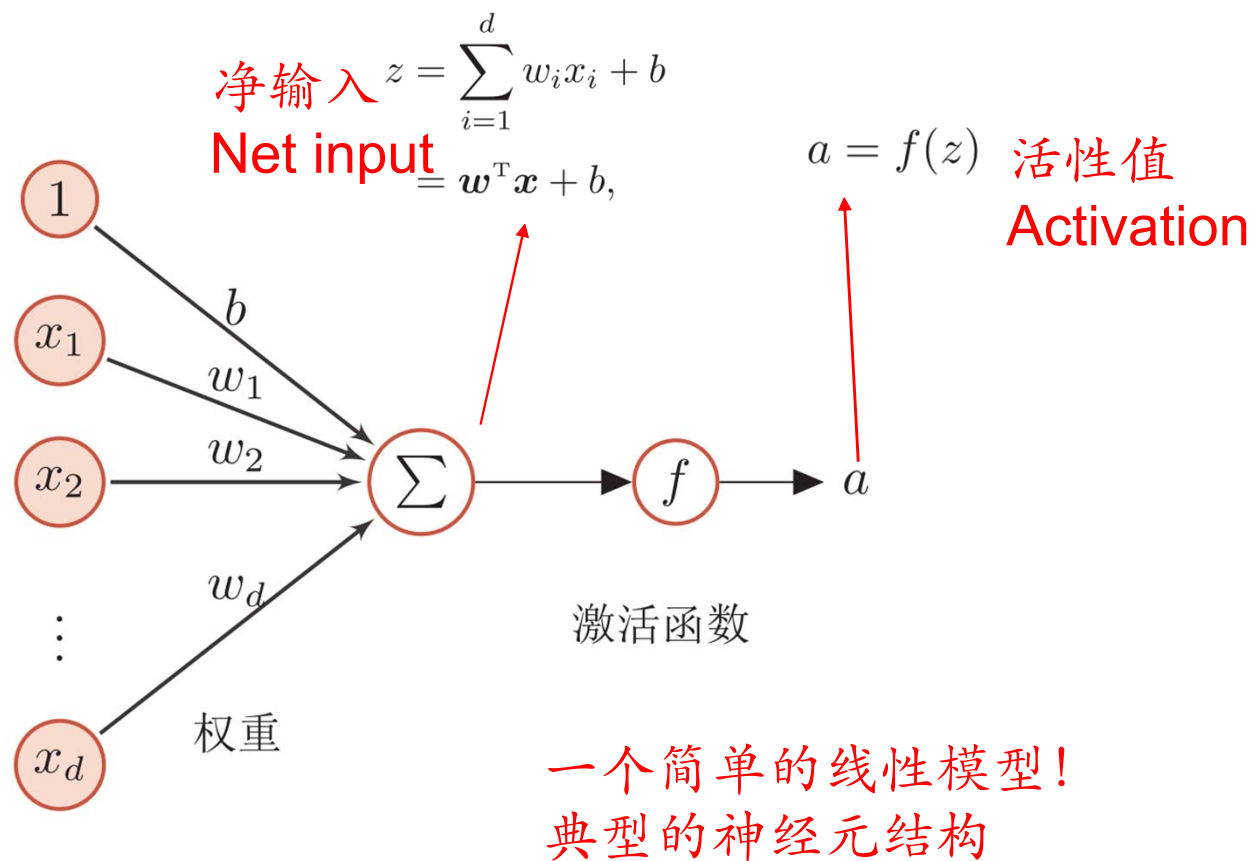
- ▶ 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：
 - ▶ 1 神经元的激活规则
 - ▶ 主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
 - ▶ 2 网络的拓扑结构
 - ▶ 不同神经元之间的连接关系。
 - ▶ 3 学习算法
 - ▶ 通过训练数据利用梯度下降算法来学习神经网络的参数。

生物神经元



单个神经细胞只有两种
状态：兴奋和抑制

人工神经元 Artificial Neuron



《神经网络与深度学习》

前馈神经网络

周晓飞
自动化学院



激活函数的性质

- ▶ 连续并可导（允许少数点上不可导）的非线性函数。
 - ▶ 可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- ▶ 激活函数及其导函数要尽可能的简单
 - ▶ 有利于提高网络计算效率。
- ▶ 激活函数的导函数的值域要在一个合适的区间内
 - ▶ 不能太大也不能太小，否则会影响训练的效率和稳定性。

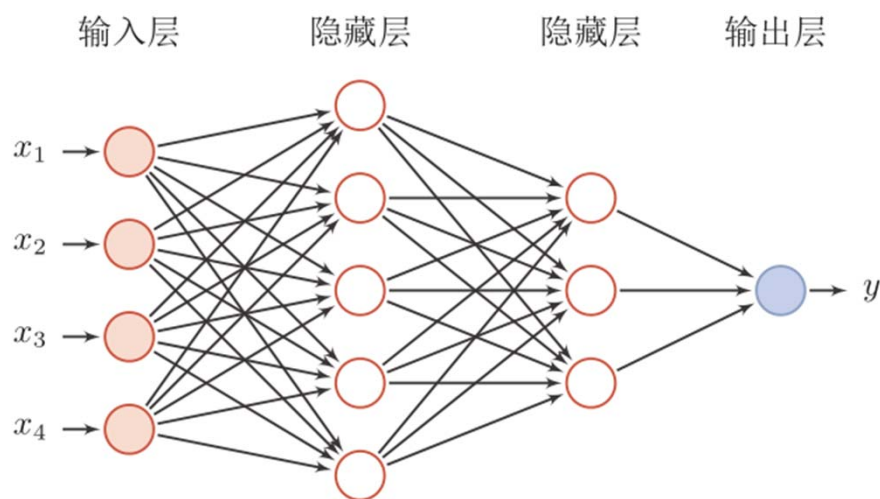
常见激活函数及其导数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

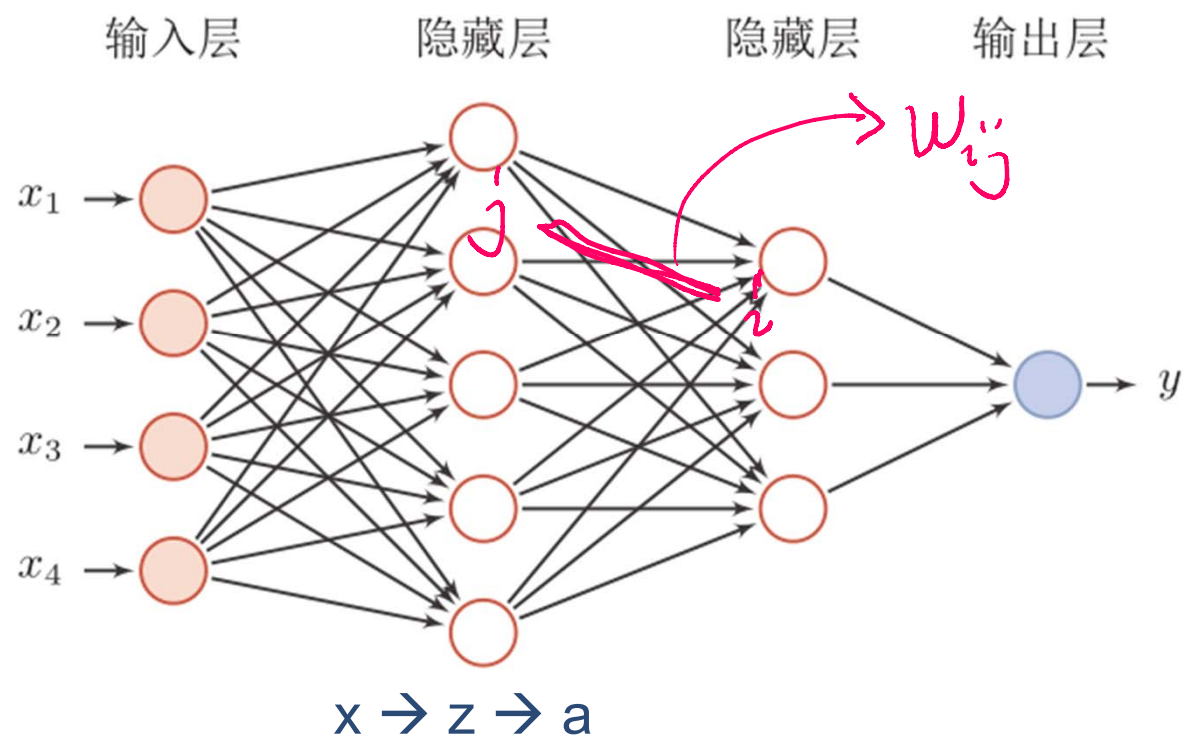
网络结构

▶ 前馈神经网络 Feedforward Neural Network FNN (全连接神经网络、多层感知器 MLP)

- ▶ 各神经元分别属于不同的层，层内无连接。
- ▶ 相邻两层之间的神经元全部两两连接。
- ▶ 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



信息传递过程



前馈网络

► 给定一个前馈神经网络，用下面的记号来描述这样网络：

- L ：表示神经网络的层数；
- n^l ：表示第 l 层神经元的个数；
- $f_l(\cdot)$ ：表示 l 层神经元的激活函数；
- $W^{(l)} \in \mathbb{R}^{n^l \times n^{l-1}}$ ：表示 $l-1$ 层到第 l 层的权重矩阵；
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n^l}$ ：表示 $l-1$ 层到第 l 层的偏置；
- $\mathbf{z}^{(l)} \in \mathbb{R}^{n^l}$ ：表示 l 层神经元的净输入（净活性值）；
- $\mathbf{a}^{(l)} \in \mathbb{R}^{n^l}$ ：表示 l 层神经元的输出（活性值）。

前馈网络

► 前馈神经网络通过下面公式进行信息传播。

$$\mathbf{z}^{(l)} = W^{(l)} \cdot \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

► 前馈计算 $\phi(\mathbf{x}; W, \mathbf{b})$:

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = f(\mathbf{x}; W, \mathbf{b})$$

如何计算梯度?

▶ 神经网络为一个复杂的复合函数

▶ 链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

▶ 反向传播算法

▶ 根据前馈网络的特点而设计的高效方法

▶ 一个更加通用的计算方法

▶ 自动微分 (Automatic Differentiation, AD)

反向传播算法 (BP)

给定一个样本 $(\mathbf{x}, \mathbf{y})$, 假设神经网络输出为 $\hat{\mathbf{y}}$, 损失函数为 $L(\mathbf{y}, \hat{\mathbf{y}})$, 采用梯度下降法需要计算损失函数关于每个参数的偏导数

计算第 l 层中参数 $\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}} \quad \text{误差项}$$

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

反向传播算法 (BP)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_i^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

计算误差项

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

- 该偏导数表示第l层神经元对最终损失的影响，也反映了最终损失对第l层神经元的敏感程度，因此一般称为第l层神经元的**误差项**

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

- 误差项也反映了不同神经元对网络能力的贡献程度，从而较好的解决了**贡献度分配问题**
CAP

按位计算

$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \end{aligned}$$

$$\begin{aligned} \delta^{(l)} &\triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \\ &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (W^{(l+1)})^T \quad \frac{\partial \mathbf{A}\mathbf{x}}{\partial \mathbf{x}} = \mathbf{A}^T$$

- 可以看出第l层的误差项可以通过第l+1层的误差项计算得到，此即是误差的反向传播
- 亦即第l层的一个神经元的误差项是所有与该神经元相连的第l+1层的神经元的误差项的权重和，然后再乘上该神经元激活函数的梯度

反向传播算法

在计算出上面三个偏导数之后, 公式 (4.49) 可以写为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}.$$

进一步, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $W^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top.$$

同理, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$

反向传播算法

计算最后一层（输出层）的误差项 $\delta^{(L)} = \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(L)}}$

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)} = \mathbf{f}_L(\mathbf{z}^{(L)}) \quad \delta^{(L)} = \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(L)}}$$

$$= \frac{\partial \mathbf{f}_L(\mathbf{z}^{(L)})}{\partial \mathbf{z}^{(L)}} \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}}$$

$$= \text{diag}(\mathbf{f}'_L(\mathbf{z}^{(L)})) \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}}$$

$$= \mathbf{f}'_L(\mathbf{z}^{(L)}) \odot \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}}$$

使用反向传播算法的随机梯度下降训练过程

算法 4.1: 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $m^{(l)}$, $1 \leq l \leq L$.

```
1 随机初始化  $W, b$  ;
2 repeat
3   对训练集  $\mathcal{D}$  中的样本随机重排序;
4   for  $n = 1 \cdots N$  do
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;
6     前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;
7     反向传播计算每一层的误差  $\delta^{(l)}$ ; // 公式 (4.63)
        // 计算每一层参数的导数
8      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$ ; // 公式 (4.65)
9      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial b^{(l)}} = \delta^{(l)}$ ; // 公式 (4.66)
        // 更新参数
10     $W^{(l)} \leftarrow W^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^T + \lambda W^{(l)})$ ;
11     $b^{(l)} \leftarrow b^{(l)} - \alpha \delta^{(l)}$ ;
12  end
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;
    输出:  $W, b$ 
```

自动梯度计算

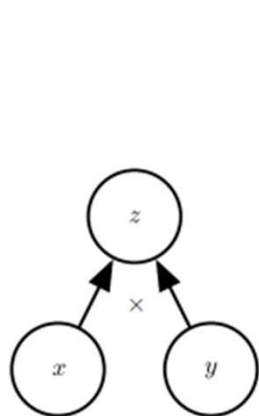
▶ 3、自动微分 (Automatic Differentiation)

- ✓ 处理对象是一个函数或一段程序
- ✓ 将数值计算分解为基本操作，包含加减乘除和一些初等函数如 $\exp/\log/\sin/\cos$ 等，然后利用链式法则来自动计算复合函数的梯度
- ✓ 介于数值微分和符号微分：数值微分一开始就代入数值近似求解，而符号微分则直接对表达式进行求解
- ✓ 灵活性高：对用户透明；无需专门的数学语言和编程；采用图的方式进行计算

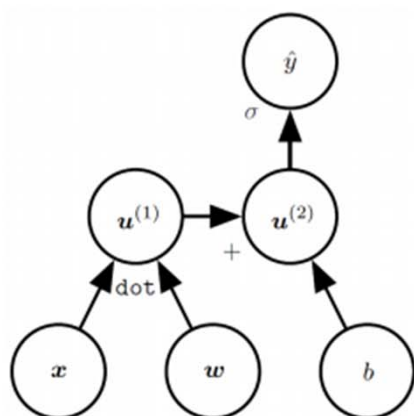
自动梯度计算

■ 计算图 Computational Graph

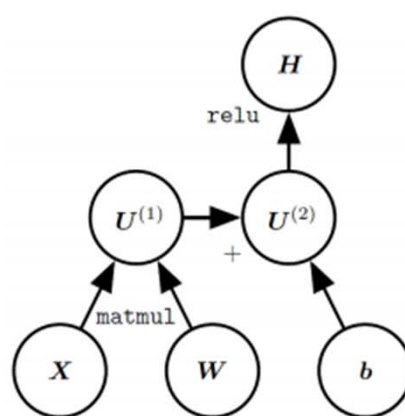
- ✓ 将复合函数分解为一系列基本操作，并以图的形式连接起来
- ✓ 数学运算的图结构表示，每个边代表一个基本操作，每个叶子节点代表一个输入变量或常量



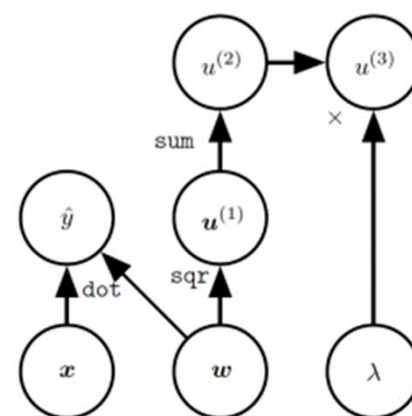
$$z = xy$$



$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$$



$$H = \max(0, \mathbf{W}\mathbf{X} + b)$$



$$\hat{y} = \mathbf{w}^T \mathbf{x}; u^{(3)} = \lambda \sum_i w_i^2$$

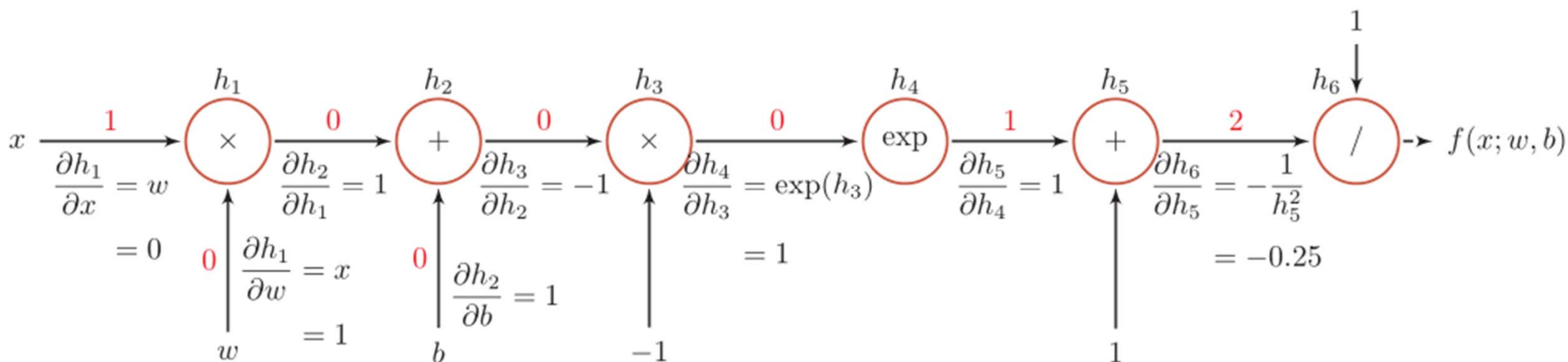
自动微分与计算图

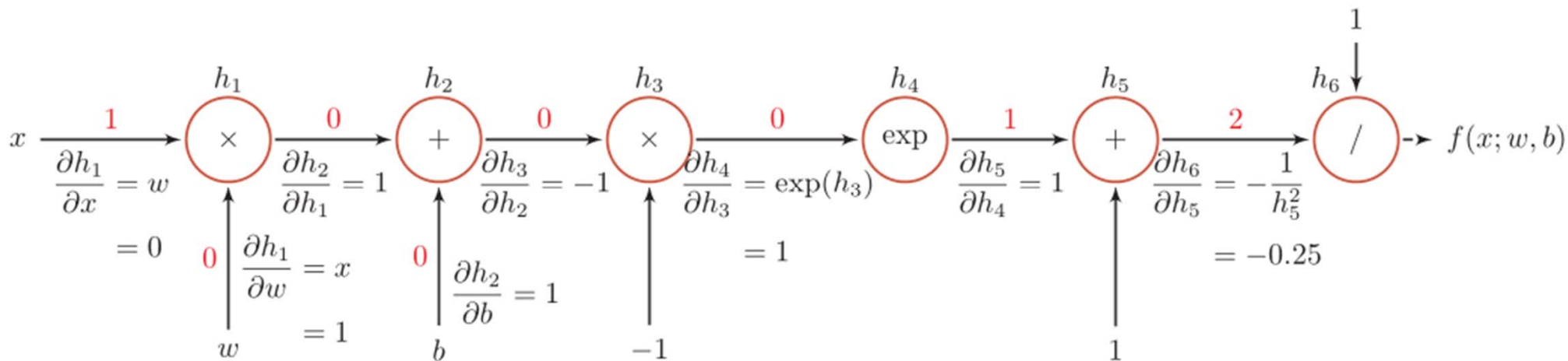
► 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

► 复合函数 $f(x; w, b)$ 的计算图

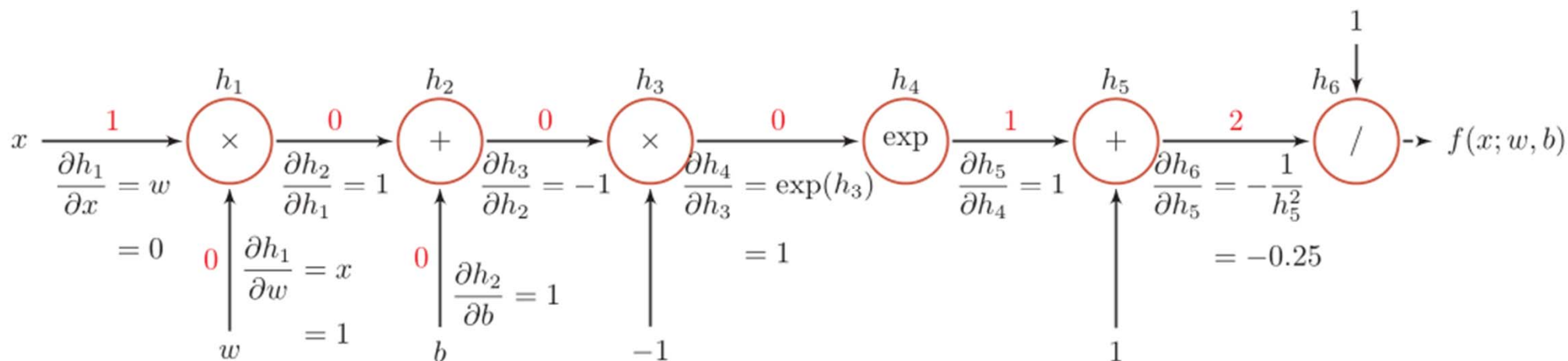
连边上的红色数字表示前向计算时复合函数中每个变量的实际取值 $x=1, w=0, b=0$





■ 表中给出了 $f(x; w, b)$ 的 6 个基本函数及其导数

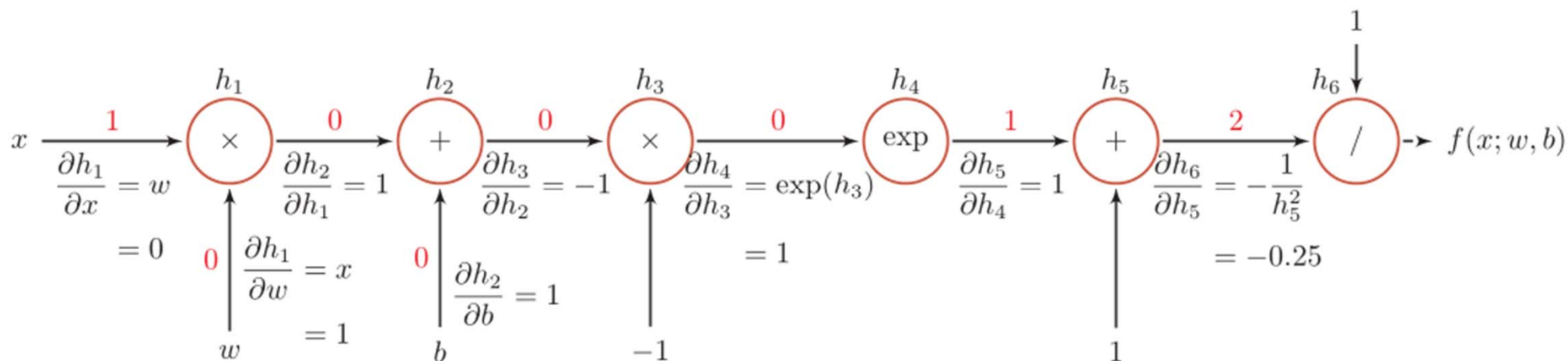
函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	



■ $f(x; w, b)$ 关于参数 w 和 b 的导数可以通过计算图上的路径上的所有导数连乘得到；

$$\frac{\partial f(x; w, b)}{\partial w} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w}$$

$$\frac{\partial f(x; w, b)}{\partial b} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial b}$$

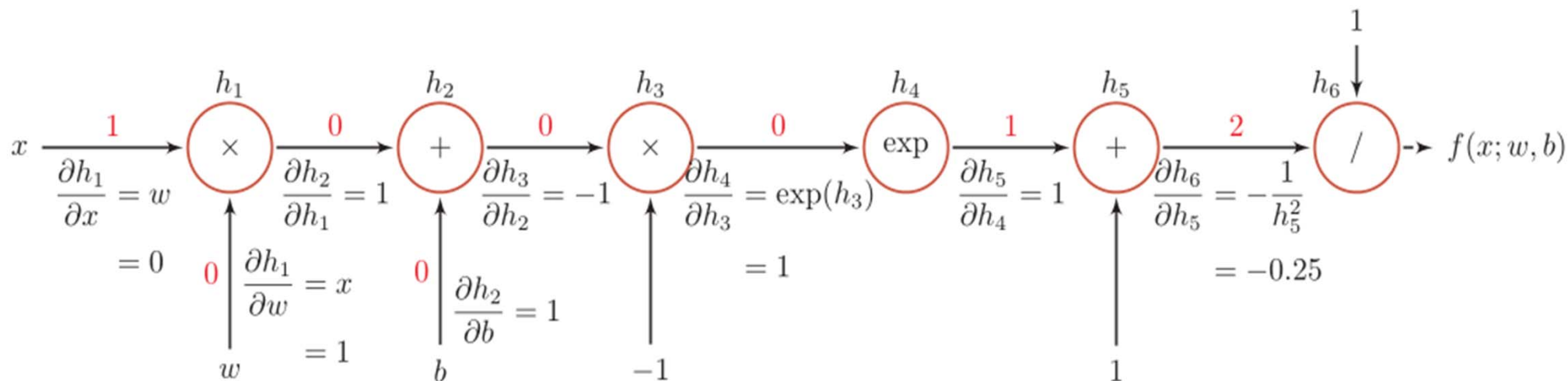


■ $f(x; w, b)$ 关于参数 w 和 b 的导数可以通过计算图上的路径上的所有导数连乘得到；

✓ 以对 w 求导为例，当 $x = 1, w = 0, b = 0$ 时，可以得到

$$\begin{aligned} \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25. \end{aligned}$$

✓ 如果函数和参数之间有多条路径，可以将这多条路径上的导数进行相加，得到最终的梯度



前向模式

$$\begin{aligned}
 \frac{\partial h_1}{\partial w} &= x = 1 \\
 \frac{\partial h_2}{\partial w} &= \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} = 1 \times 1 = 1 \\
 \frac{\partial h_3}{\partial w} &= \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w} = -1 \times 1 \\
 &\vdots \\
 \frac{\partial h_6}{\partial w} &= \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial w} = -0.25 \times -1 = 0.25 \\
 \frac{\partial f(x; w, b)}{\partial w} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial w} = 1 \times 0.25 = 0.25
 \end{aligned}$$

网络

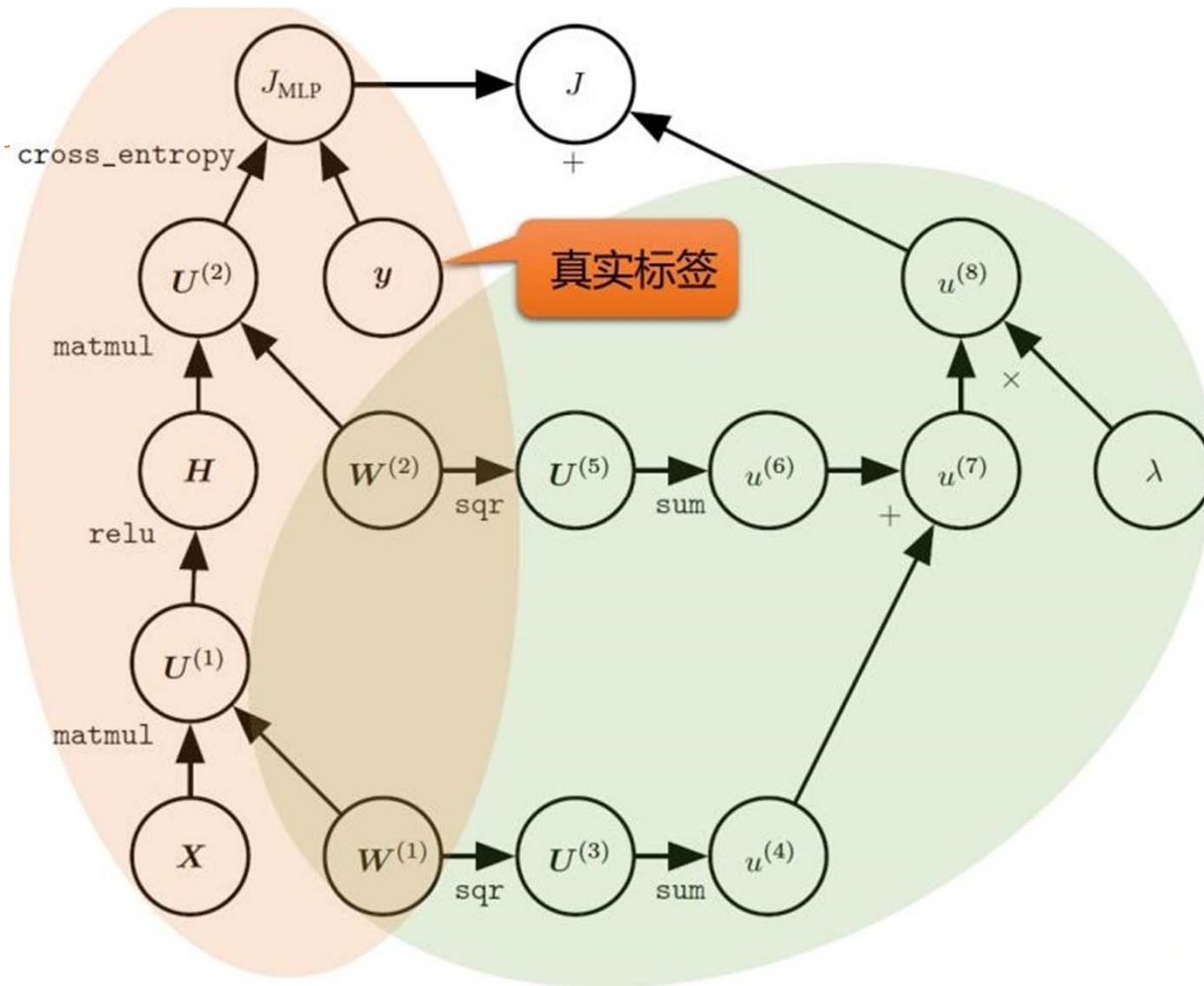
反向模式

$$\begin{aligned}
 \frac{\partial f(x; w, b)}{\partial h_6} &= 1 \\
 \frac{\partial f(x; w, b)}{\partial h_5} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} = 1 \times -0.25 \\
 \frac{\partial f(x; w, b)}{\partial h_4} &= \frac{\partial f(x; w, b)}{\partial h_5} \frac{\partial h_5}{\partial h_4} = -0.25 \times 1 = -0.25 \\
 &\vdots \\
 \frac{\partial f(x; w, b)}{\partial w} &= \frac{\partial f(x; w, b)}{\partial h_1} \frac{\partial h_1}{\partial w} = 0.25 \times 1 = 0.25
 \end{aligned}$$

计算图实例

正则化单隐层MLP计算图

$$J = J_{\text{MLP}} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$



引用

<https://rmcong.github.io/DLCourse/chapter4.pdf> 《神经网络与深度学习》

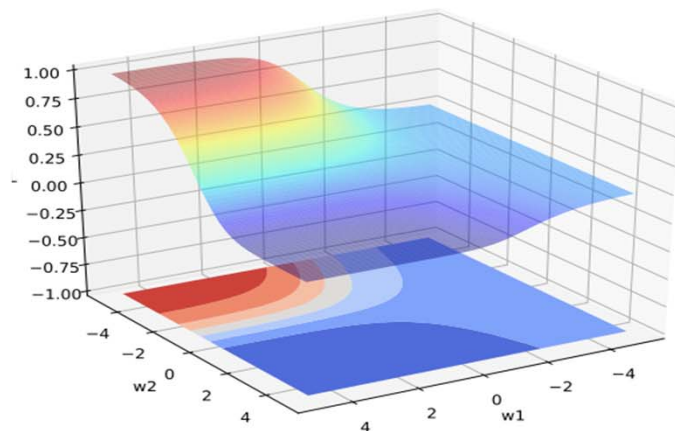
优化问题

► 非凸优化问题

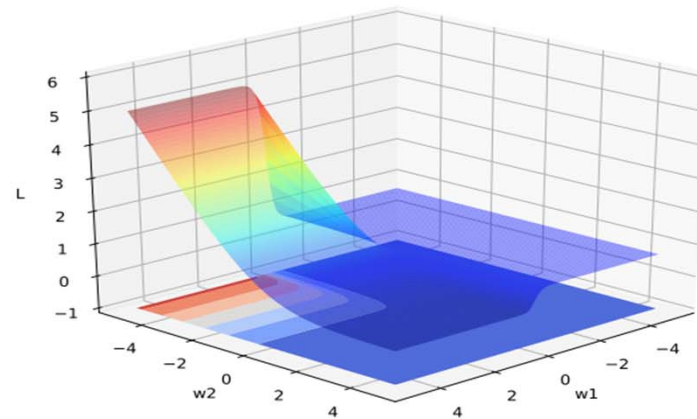
以最简单的1-1-1结构的两层神经网络为例 $y = \sigma(w_2 \sigma(w_1 x))$,

w_1 和 w_2 是参数，则输入样本 $(x,y)=(1,1)$ ，对应的平方误差损失和交叉熵损失与参数的关系如下图

可以看出两种损失函数都是关于参数的非凸函数：即存在局部最优而非全局最优解，影响迭代



(a) 平方误差损失



(b) 交叉熵损失

图 4.9 神经网络 $y = \sigma(w_2 \sigma(w_1 x))$ 的损失函数

优化问题

► 梯度消失问题 (Vanishing Gradient Problem) 梯度弥散问题

► 误差反向传播算法

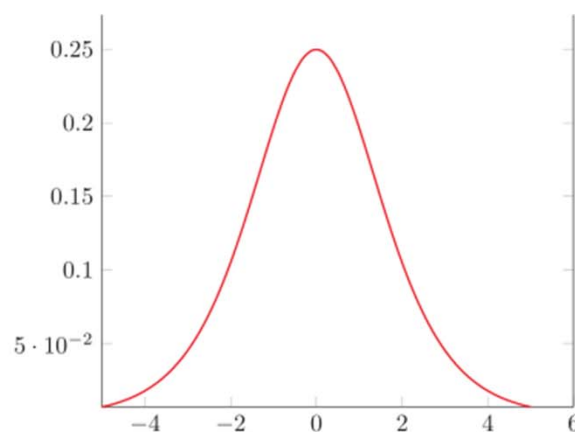
$$\delta^{(l)} = \boxed{f'_1(z^{(l)})} \odot \left(\mathbf{W}^{(l+1)} \right)^T \delta^{(l+1)}$$

► 误差反向传播时，在每一层都要乘以该层激活函数的导数

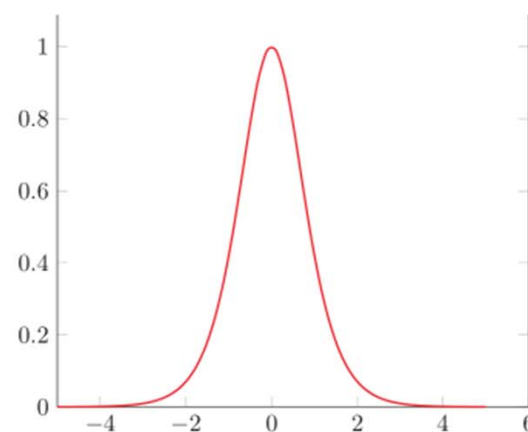
► 当采用sigmoid型激活函数时，导数为

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \in [0, 0.25]$$

$$\tanh'(x) = 1 - (\tanh(x))^2 \in [0, 1]$$



(a) logistic 函数的导数



(b) tanh 函数的导数

优化问题

► 梯度消失问题 (Vanishing Gradient Problem) 梯度弥散问题

- 当采用sigmoid型激活函数时，导数为

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \in [0, 0.25]$$

$$\tanh'(x) = 1 - (\tanh(x))^2 \in [0, 1]$$

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

- 由于Sigmoid型函数的饱和性，饱和区导数趋近于0，如此，导数经过每一层都会衰减，随着层数的加深，造成了梯度消失，使得整个网络很难训练，这就是所谓的**梯度消失问题**，亦称**梯度弥散问题**

《神经网络与深度学习》

卷积神经网络 Convolutional Neural Network, CNN

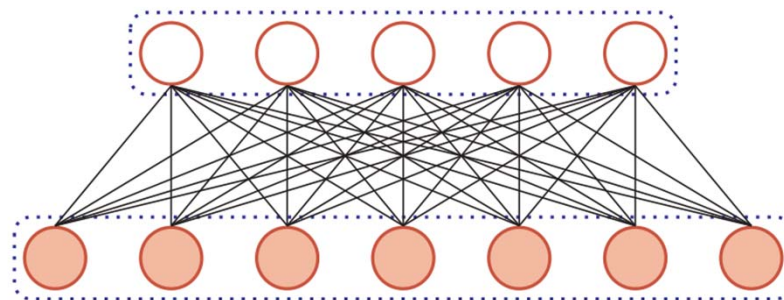


周晓飞
自动化学院

全连接前馈神经网络 (FCNN)

▶1、权重矩阵的参数非常多

- ▶ 导致训练效率低，易出现过拟合



▶2、局部不变性特征

- ▶ 自然图像中的物体都具有局部不变性特征，比如尺度缩放、平移、旋转等操作不影响其语义信息。
- ▶ 全连接前馈网络很难提取这些局部不变特征（需配合数据增强来提高性能）。

卷积神经网络

- ▶ 卷积神经网络（Convolutional Neural Networks, CNN）是一种前馈神经网络。
 - ▶ 卷积神经网络是受生物学上**感受野（Receptive Field）**的机制而提出的。
 - ▶ 视觉神经系统中，神经元的**感受野**是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元，即神经元只接受其所支配的刺激区域内的信号。
- ▶ 目前的卷积神经网络一般是由**卷积层、汇聚层和全连接层**交叉堆叠而成的前馈神经网络
- ▶ 卷积神经网络结构上的**三个特性**：
 - ▶ 局部连接
 - ▶ 权重共享
 - ▶ 池化（汇聚）：空间或时间上的次采样



三特性使得卷积神经网络具有一定程度上的**平移、缩放和旋转不变性**。

二维卷积

► 输入信息X和滤波W的二维卷积定义为

$$Y = W * X$$

► 其中*表示二维卷积运算。

► 下图为卷积示例（卷积核旋转180°）

1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

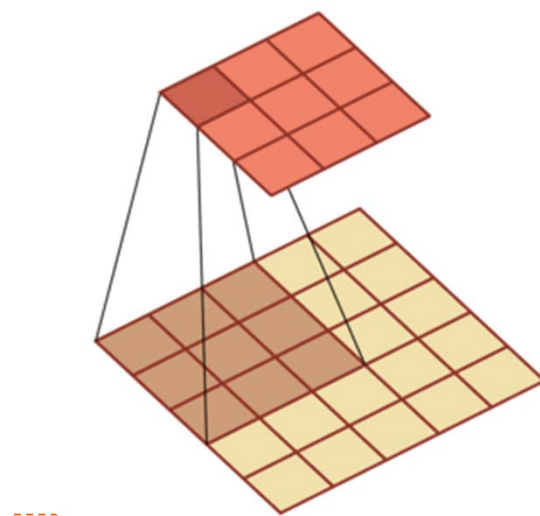
⊗

1	0	0
0	0	0
0	0	-1

=

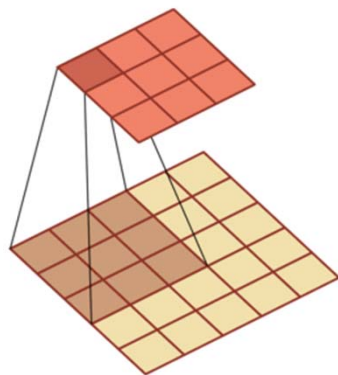
0	-2	-1
2	2	4
-1	0	0

（Note: In the original image, the first matrix has blue annotations: the top row of the 3x3 sub-region is multiplied by -1, the middle row by 0, and the bottom row by 0. The bottom row of the 3x3 sub-region is multiplied by 0, the middle row by 0, and the bottom row by 1. The 3x3 sub-region is highlighted in light red.)

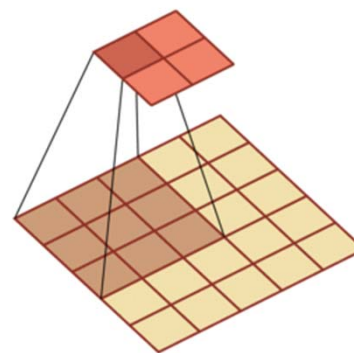


二维卷积

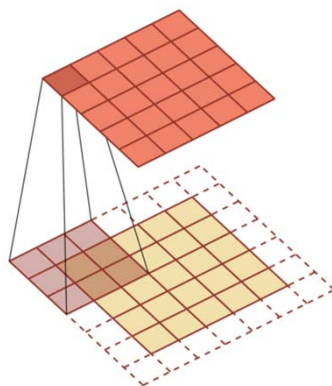
$$W^* = \frac{W - K + 2P}{S} + 1$$



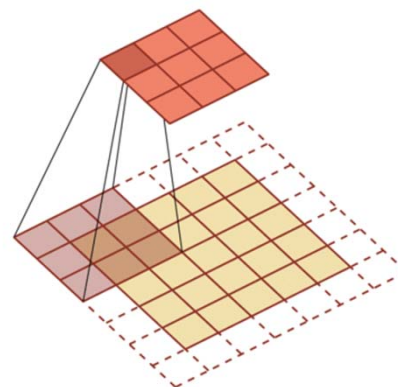
步长1，零填充0



步长2，零填充0



步长1，零填充1



步长2，零填充1

互相关 (Cross-Correlation)

- ▶ 在机器学习和图像处理领域，卷积的主要功能是在一个图像（或某种特征）上滑动一个卷积核（滤波器），通过卷积操作提取特征。
- ▶ 在计算卷积的过程中，需要进行卷积核翻转。但在具体实现上，一般会以互相关操作来代替卷积，从而减少一些不必要的操作或开销。
- ▶ 互相关是一个衡量两个序列相关性的函数，通常是用滑动窗口的点积计算来实现。给定一个图像 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$ 它们的相关性为：

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i+u-1, j+v-1}$$

互相关

互相关

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i+u-1, j+v-1}$$

协相关：是把核与输入数据对应相乘再求和

2	10	8
5	15	20
6	11	9

 $\otimes$

1	8
-5	6

 $=$

2 ¹	10 ⁸	8
5 ⁻⁵	15 ⁶	20
6	11	9

 $=$

147	119
161	174

卷积

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{i-u+1, j-v+1}$$

卷积：是把核先反转180，再作协相关

2	10	8
5	15	20
6	11	9

 $\star$

1	8
-5	6

 $\Updownarrow$

2	10	8
5	15	20
6	11	9

 $\otimes$

6	-5
8	1

 $=$

2 ⁶	10 ⁻⁵	8
5 ⁸	15 ¹	20
6	11	9

 $=$

17	160
14	87

卷积神经网络-卷积层

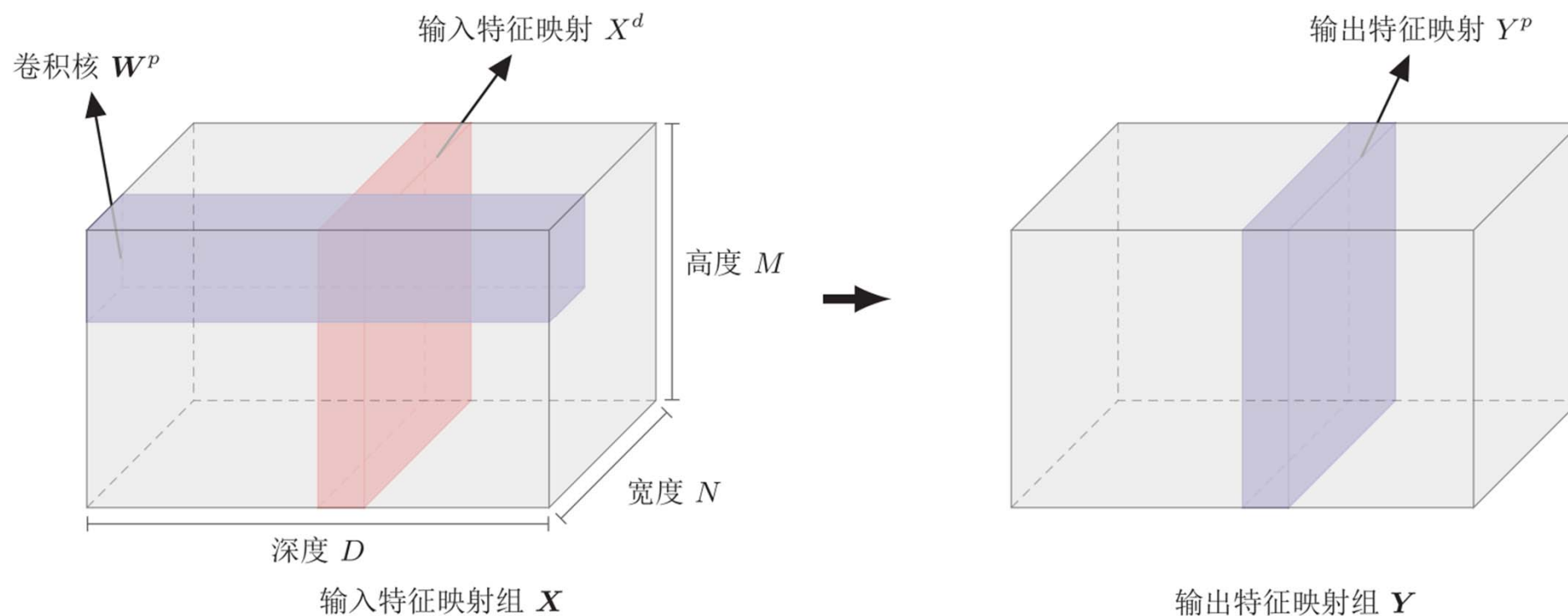
- ▶ **特征映射 Feature Map**为一幅图像经过卷积提取到的特征，
每个特征映射可以作为一种卷积抽取的图像特征。为了提高卷积网络的表示能力，可以在每一层使用多个不同的特征映射，以更好地表示图像的特征。
- ▶ 假设一个卷积层的结构如下：
 - ▶ 输入特征映射组： $\chi \in \mathbb{R}^{M \times N \times D}$ ，每个切片Slice矩阵 $\mathbf{X}^d \in \mathbb{R}^{M \times N}$ 表示一个输入特征映射 $1 \leq d \leq D$ ；
 - ▶ 输出特征映射组： $y \in \mathbb{R}^{M' \times N' \times P}$ ，每个切片 $\mathbf{Y}^p \in \mathbb{R}^{M' \times N'}$ 为一个输出特征映射， $1 \leq p \leq P$
 - ▶ 卷积核： $\mathbf{W} \in \mathbb{R}^{U \times V \times P \times D}$ ，每个切片 $\mathbf{W}^{p,d} \in \mathbb{R}^{U \times V}$ 为一个二维卷积核 $1 \leq d \leq D, 1 \leq p \leq P$

卷积核 $U \times V \times P \times D$ P组Filter 1组Filter有D个Filter Filter=卷积核

卷积神经网络-卷积层

► 典型的卷积层为3维结构

卷积核深度应该与输入一致D



卷积神经网络-卷积层

- 为了计算输出特征 $\mathbf{Y}^p$ ，用卷积核 $\mathbf{W}^{p,1}, \mathbf{W}^{p,2}, \dots, \mathbf{W}^{p,D}$ 分别对输入特征 $\mathbf{X}^1, \mathbf{X}^2, \dots, \mathbf{X}^D$ 进行卷积，然后将卷积结果相加，并加上一个标量偏置 b ，得到卷积层的净输入 $\mathbf{Z}^p$ ，再经过非线性激活函数 $f(\cdot)$ 后得到输出特征映射 $\mathbf{Y}^p$ 。

$$\mathbf{Z}^p = \mathbf{W}^p \otimes \mathbf{X} + b^p = \sum_{d=1}^D \mathbf{W}^{p,d} \otimes \mathbf{X}^d + b^p,$$

$$\mathbf{Y}^p = f(\mathbf{Z}^p).$$

- 其中 $\mathbf{W}^p \in \mathbb{R}^{U \times V \times D}$ 为三维卷积核

卷积神经网络-卷积层

整个计算过程如图所示

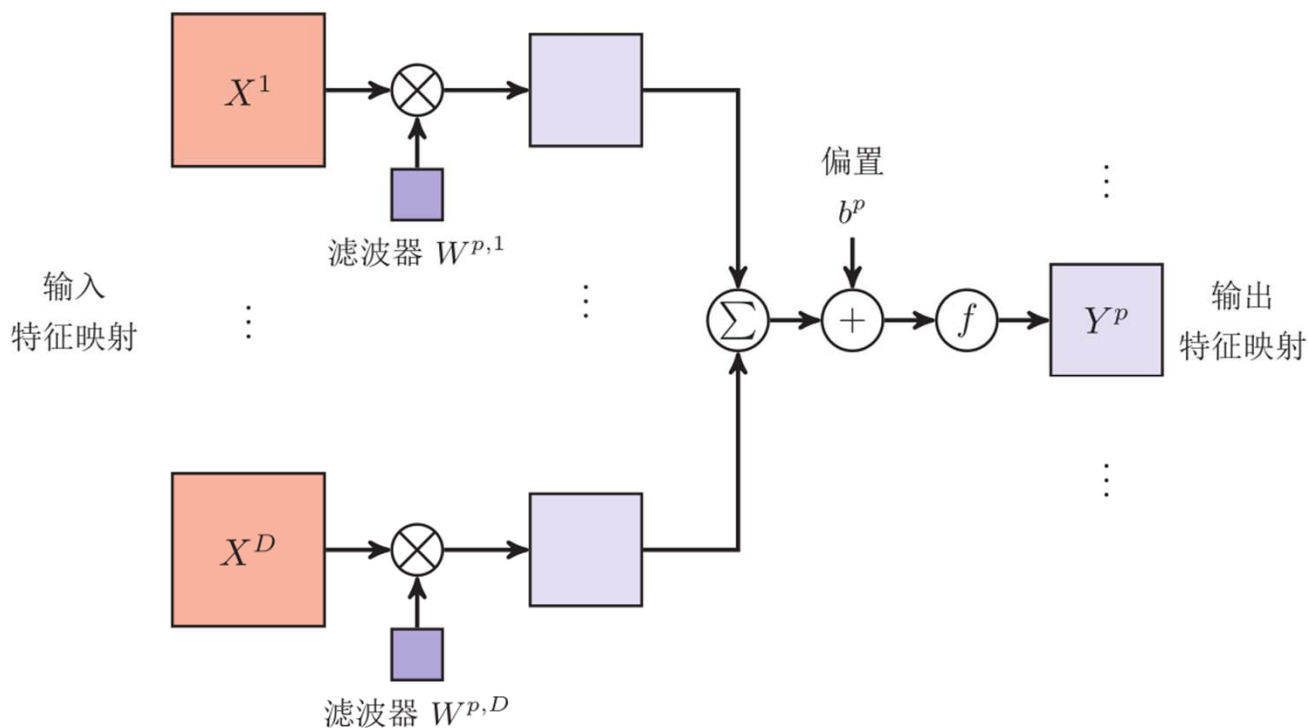
在输入为 $\chi \in \mathbb{R}^{M \times N \times D}$ ，输出为 $y \in \mathbb{R}^{M' \times N' \times P}$ 的卷积层中，每一个输出特征映射都需要D个卷积核以及一个偏置。假设每个卷积核的大小为 $U \times V$ ，那么共需要：

$P \times D \times (U \times V) + P$ 个参数

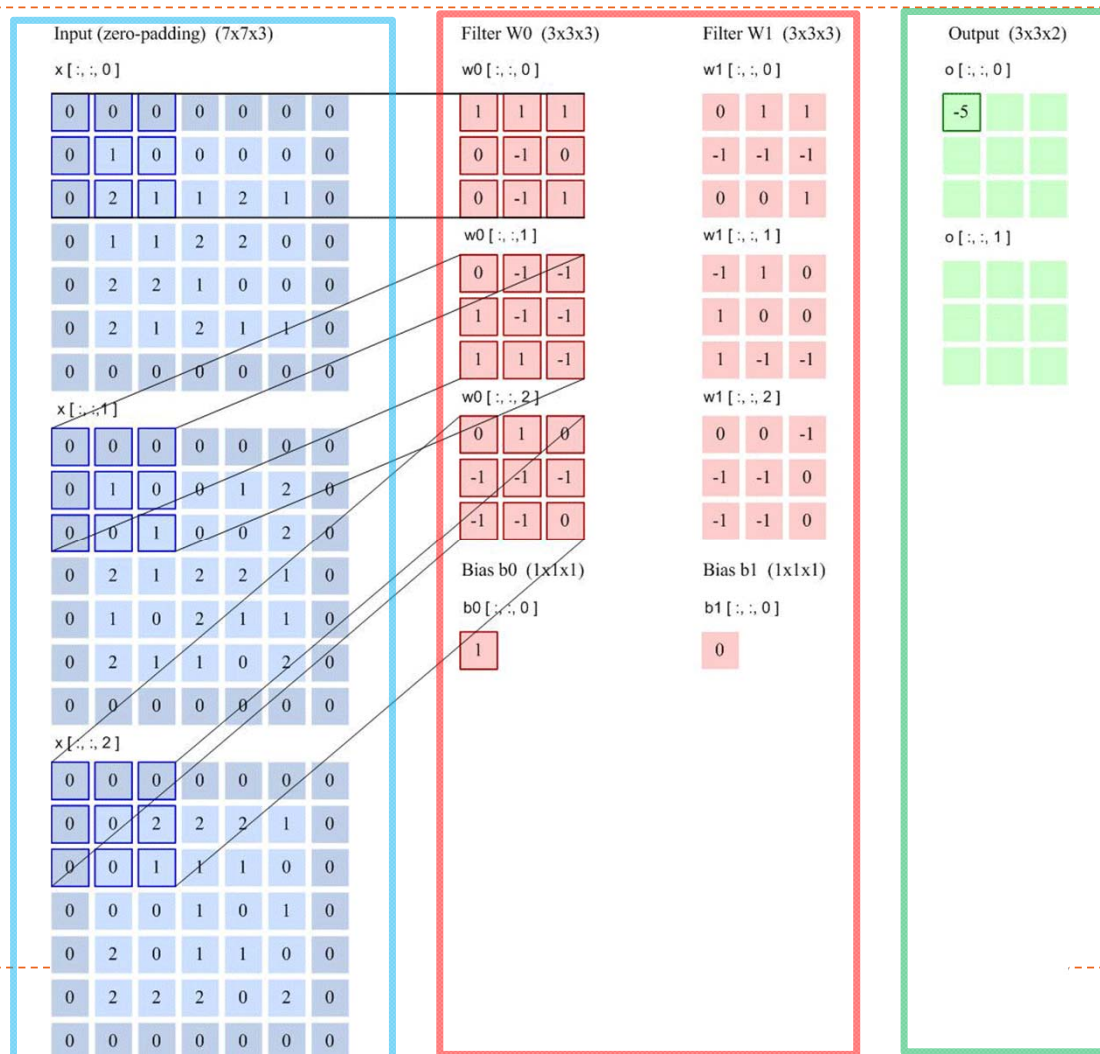
(空间复杂度)

$$Z^p = \mathbf{W}^p \otimes \mathbf{X} + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p,$$

$$Y^p = f(Z^p).$$



步长2 filter个数3 3*3 填充



卷积到底是在做一件怎么样的事情，任意给你一张图片你应该知道卷积是在做什么？⁵³

卷积神经网络

► 汇聚层 (pooling)
其作用是进

► 假设汇聚层
将其划分为
，也可以不
，作为这个
► 对每一个输
汇聚层输入

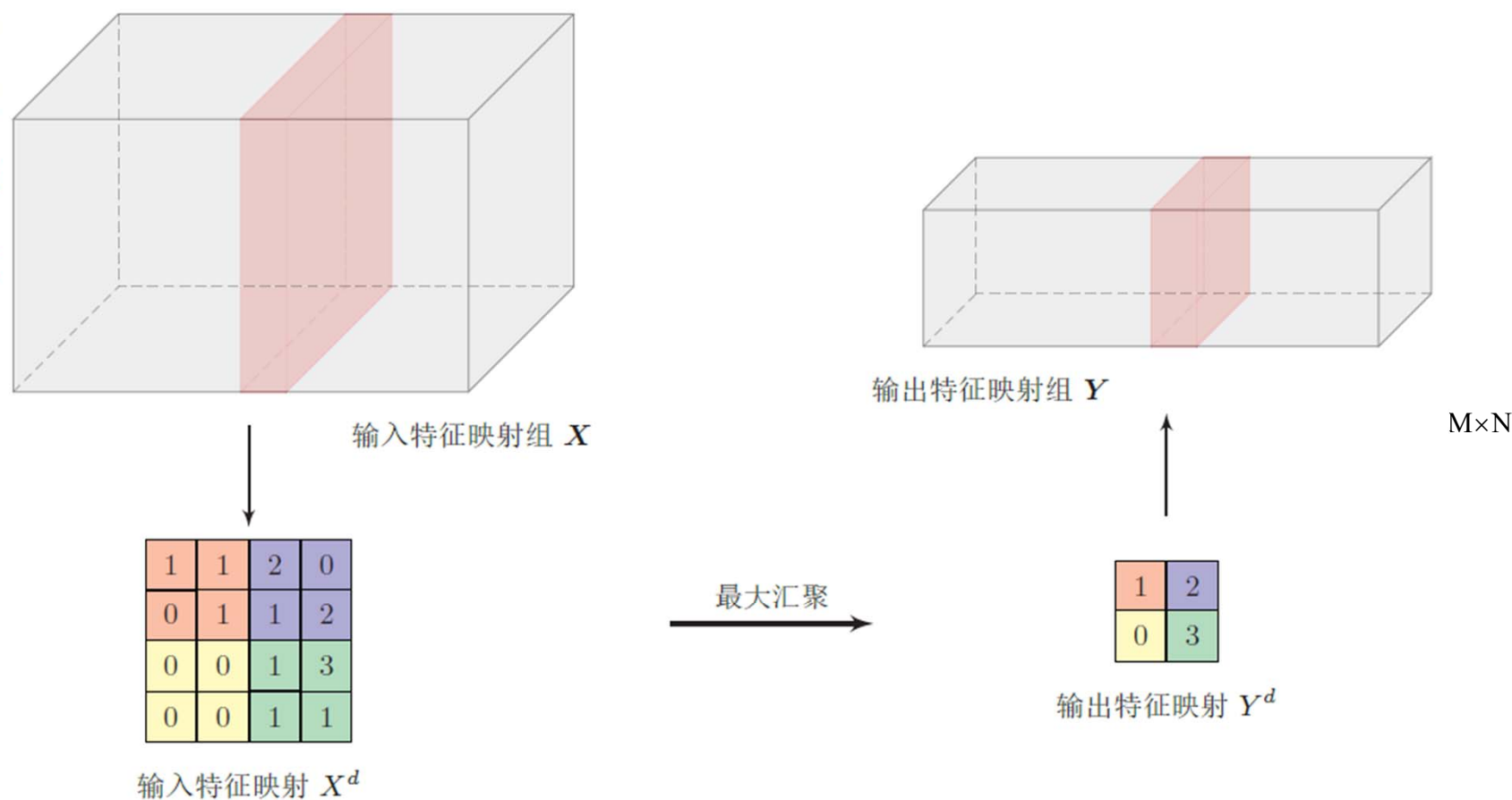
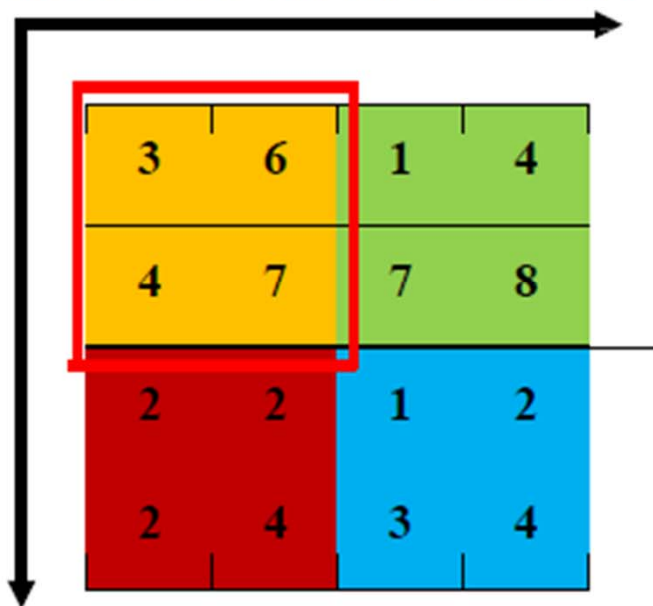


图 5.8 汇聚层中最大汇聚过程示例

卷积神经网络-汇聚层

▶ 常见的汇聚有两种：

Kernel size: 2x2 Stride:2



Max Pooling



7	8
4	4

Mean Pooling



5	5
2.5	2.5

转置卷积(Transposed Convolution/Deconvolution)

- ▶ 卷积操作也可以写成仿射变换。假设一个5维向量 $\mathbf{x}$ ，经过大小为3的卷积核 $\mathbf{w}=[w_1, w_2, w_3]^T$ 进行卷积，得到3维向量 $\mathbf{z}$ ，
- ▶ 同时，要实现 $\mathbf{z}$ 到 $\mathbf{x}$ 的映射，可转置仿射矩阵 $\mathbf{C}$

$$\mathbf{z} = \mathbf{w} \otimes \mathbf{x}$$

$$= \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix} \mathbf{x}$$

$$= \mathbf{C} \mathbf{x}$$

$$\mathbf{x} = \mathbf{C}^T \mathbf{z}$$

$$= \begin{bmatrix} w_1 & 0 & 0 \\ w_2 & w_1 & 0 \\ w_3 & w_2 & w_1 \\ 0 & w_3 & w_2 \\ 0 & 0 & w_3 \end{bmatrix} \mathbf{z} = \text{rot180}(\mathbf{w}) \tilde{\otimes} \mathbf{z}$$

转置卷积(Transposed Convolution/Deconvolution)

- ▶ 从上述两式可以看出，从仿射变换的角度来看两个卷积操作也是形式上的转置关系 $\mathbf{z} = \mathbf{w} \otimes \mathbf{x}$ 和 $\mathbf{x} = \text{rot180}(\mathbf{w}) \otimes \mathbf{z}$ ；因此，将低维特征映射到高维特征的卷积操作称为转置卷积 (Transposed Convolution)，或反卷积 (Deconvolution)
- ▶ 在卷积网络中，卷积层的前向计算和反向传播也是一种转置关系。对于 M 维向量 $\mathbf{z}$ 和大小为 K 的卷积核，如果希望通过卷积操作来映射到更高维的向量，只需要对向量 $\mathbf{z}$ 进行两端补零 $P=K-1$ ，然后进行卷积，可以得到 $M+K-1$ 维的向量。

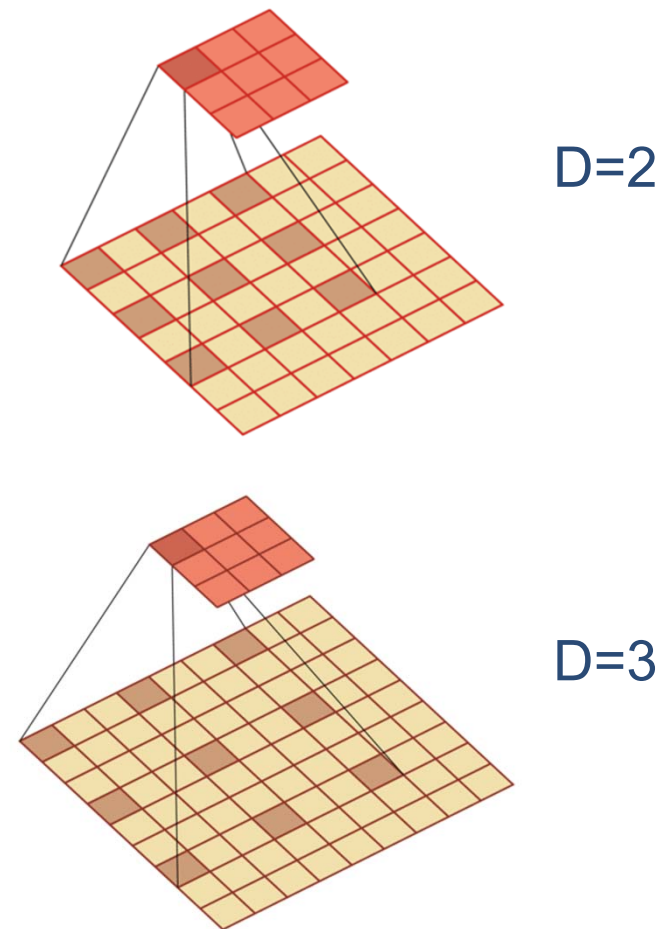
空洞卷积(Atrous Convolution/Dilated Convolution)

▶ 如何增加输出单元的感受野

- ▶ 增加卷积核的大小
- ▶ 增加层数来实现
- ▶ 在卷积之前进行汇聚操作

▶ 空洞卷积

- ▶ 通过给卷积核插入“空洞”来变相地增加其大小。



空洞卷积(Atrous Convolution/Dilated Convolution)

- ▶ **空洞卷积** (Atrous convolution) : 是一种不增加参数数量, 同时增加输出单元感受野的方法, 也称为膨胀卷积 (Dilated convolution)
- ▶ 空洞卷积通过给**卷积核**插入“空洞”来变相的增加其大小, 如果在卷积核的每两个元素之间插入D-1个空洞, 卷积核有效大小为:

$$K' = K + (K - 1) \times (D - 1)$$

- ▶ 其中D为膨胀率 (Dilation Rate) 。当D=1时, 卷积核为普通卷积核。

Inception网络

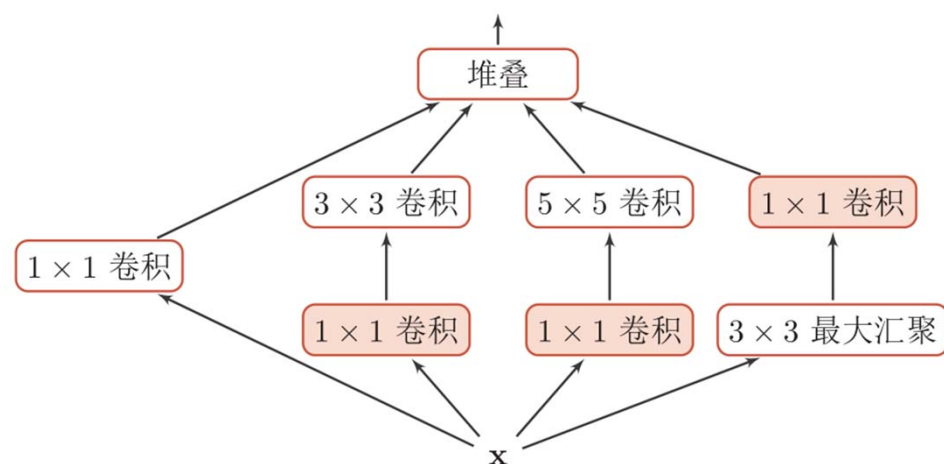
► Inception module 包含四个分支

✓ **Shortcut 连接**：将前一层输入通过 1×1 卷积

✓ **多尺度滤波**：输入通过 1×1 卷积之后分别连接卷积核大小为 3 和 5 的卷积

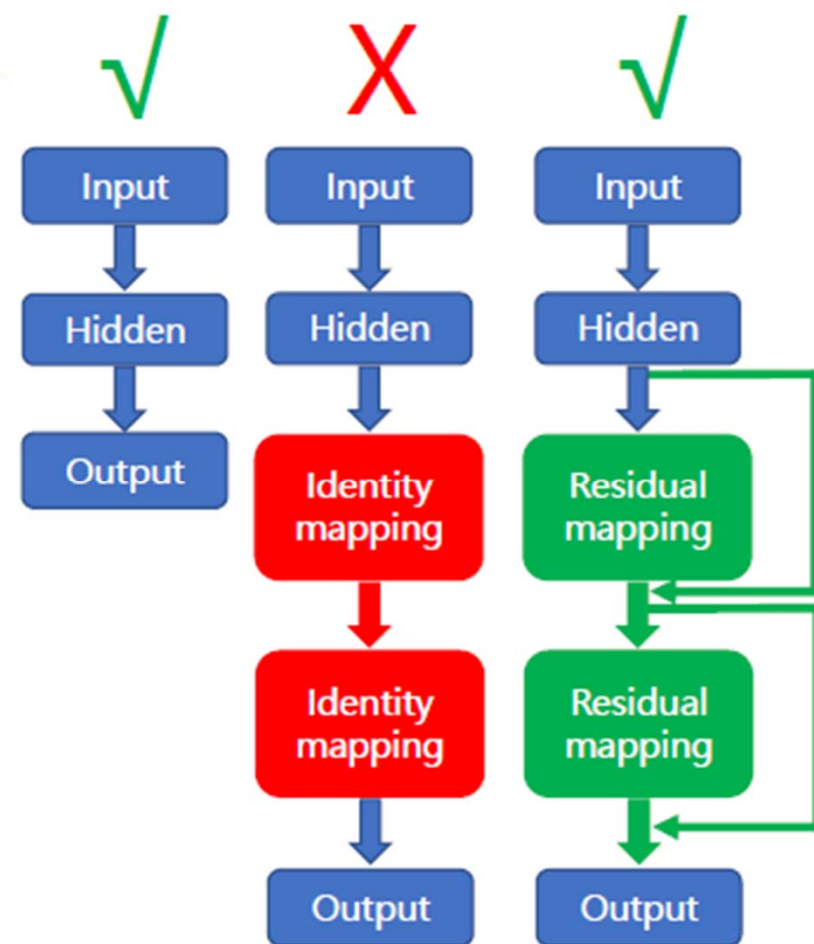
✓ **池化分支**：相继连接 3×3 pooling 和 1×1 卷积

■ 四个分支的输出经过串联恢复到输入通道大小



残差网络 (Residual Network, ResNet)

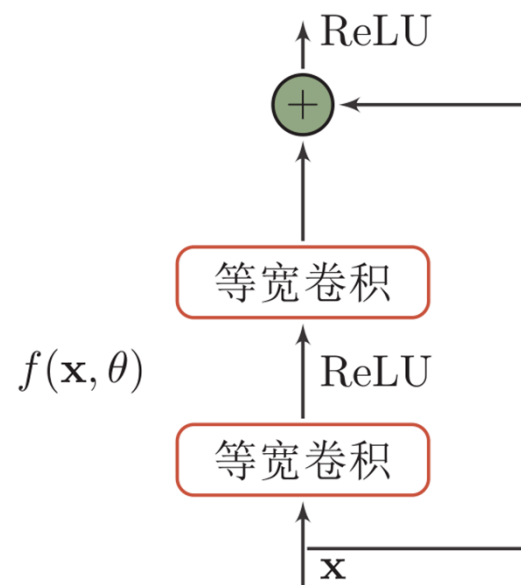
- ▶ 简单的层数堆叠不能提升网络性能，如何利用网络加深带来的优势呢？
- ▶ 实验表明，通过添加恒等映射不能提高网络准确率，因此，网络对恒等映射的逼近存在困难。
- ▶ 相比于恒等映射，网络对恒等映射附近扰动学习更加简单。



残差网络

- ▶ **残差网络** (Residual Network, ResNet) 是通过给非线性的卷积层增加**直连边**的方式来提高信息的传播效率。
- ▶ 假设在一个深度网络中，我们期望一个非线性单元（可以为一层或多层的卷积层） $f(\mathbf{x}, \theta)$ 去逼近一个目标函数为 $h(\mathbf{x})$
- ▶ 将目标函数拆分成两部分：**恒等函数**和**残差函数**

$$h(\mathbf{x}) = \underbrace{\mathbf{x}}_{\text{恒等函数}} + \underbrace{(h(\mathbf{x}) - \mathbf{x})}_{\text{残差函数}} \quad \xrightarrow{\text{red arrow}} \quad f(\mathbf{x}, \theta)$$

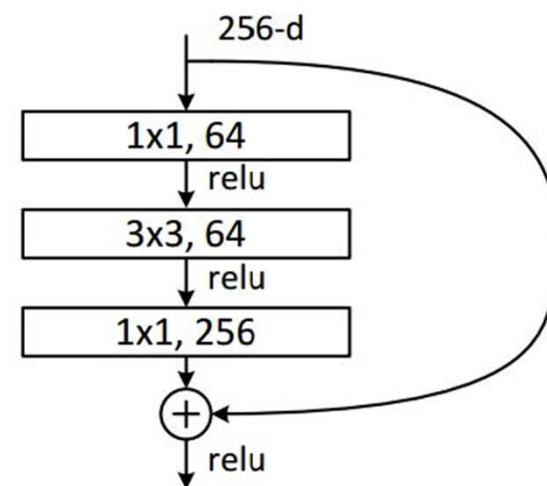
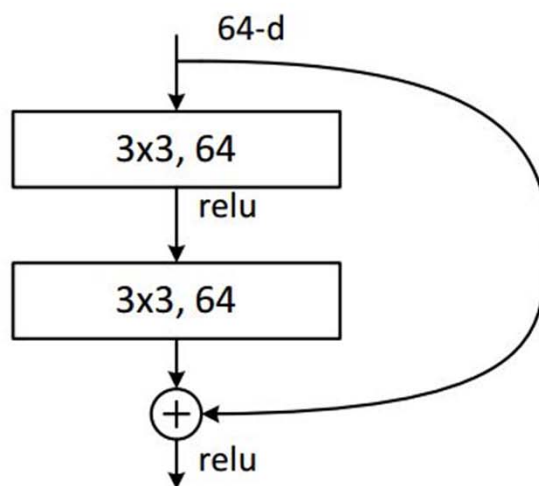
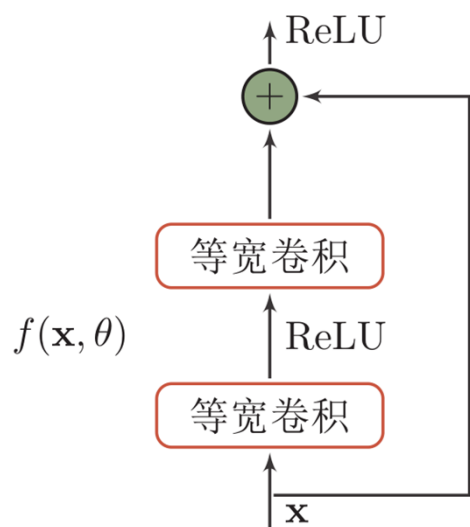


残差单元

■ 用三层连接代替两层 (1×1 , 3×3 , 1×1 卷积

✓ 1×1 : 降维和升维

✓ 3×3 : 具有较小输入输出维度的卷积



✓ 没有带来额外的参数和计算开销。

✓ 便于和具有相同结构的“平常”网络进行对比。

《神经网络与深度学习》

循环神经网络



周晓飞
自动化学院

简单循环神经网络 Simple Recurrent Network

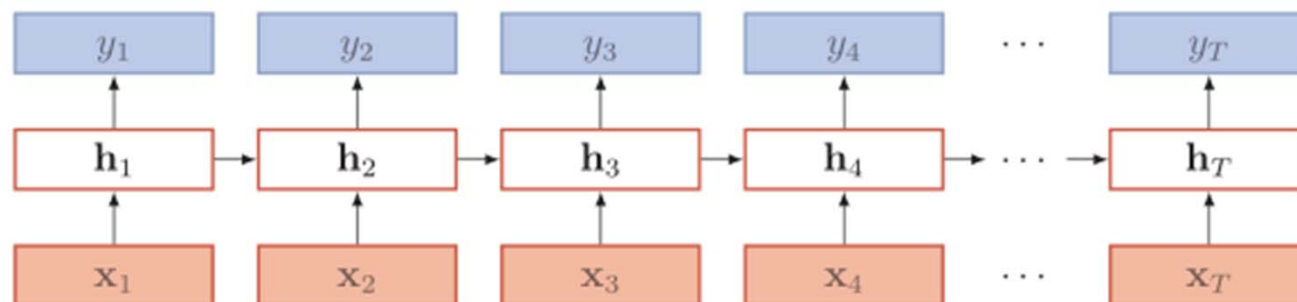
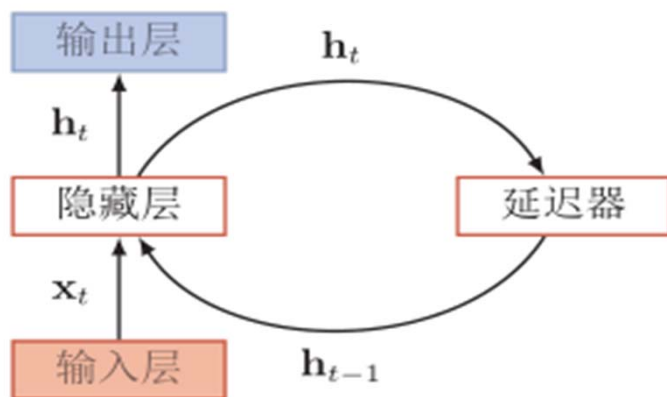
▶ 简单循环神经网络 Simple Recurrent Network, SRN

- ▶ 只有一个隐藏层的神经网络；增加了从隐藏层到隐藏层的反馈连接
- ▶ 令 $\mathbf{x}_t \in \mathbb{R}^M$ 表示在时刻 t 时网络的输入， $\mathbf{h}_t \in \mathbb{R}^D$ 表示隐藏层的状态（即隐藏层神经元活性值），则 $\mathbf{h}_t$ 不仅和当前时刻的输入 $\mathbf{x}_t$ 相关，也和上一时刻的隐状态 $\mathbf{h}_{t-1}$ 相关。简单循环网络在时刻 t 的更新公式为

$$\begin{aligned} \mathbf{z}_t &= \mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{x}_t + \mathbf{b} \\ \mathbf{h}_t &= f(\mathbf{z}_t) \end{aligned} \quad \Rightarrow \quad \mathbf{h}_t = f(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{x}_t + \mathbf{b})$$

按时间展开

- 如果我们把每个时刻的状态都看作前馈神经网络的一层，循环神经网络可以看作在时间维度上权值共享的神经网络。



应用到机器学习

- ▶ 序列到类别
- ▶ 同步的序列到序列模式
- ▶ 异步的序列到序列模式

1 序列到类别模式

► 情感分类

带着愉悦的心情
看了这部电影

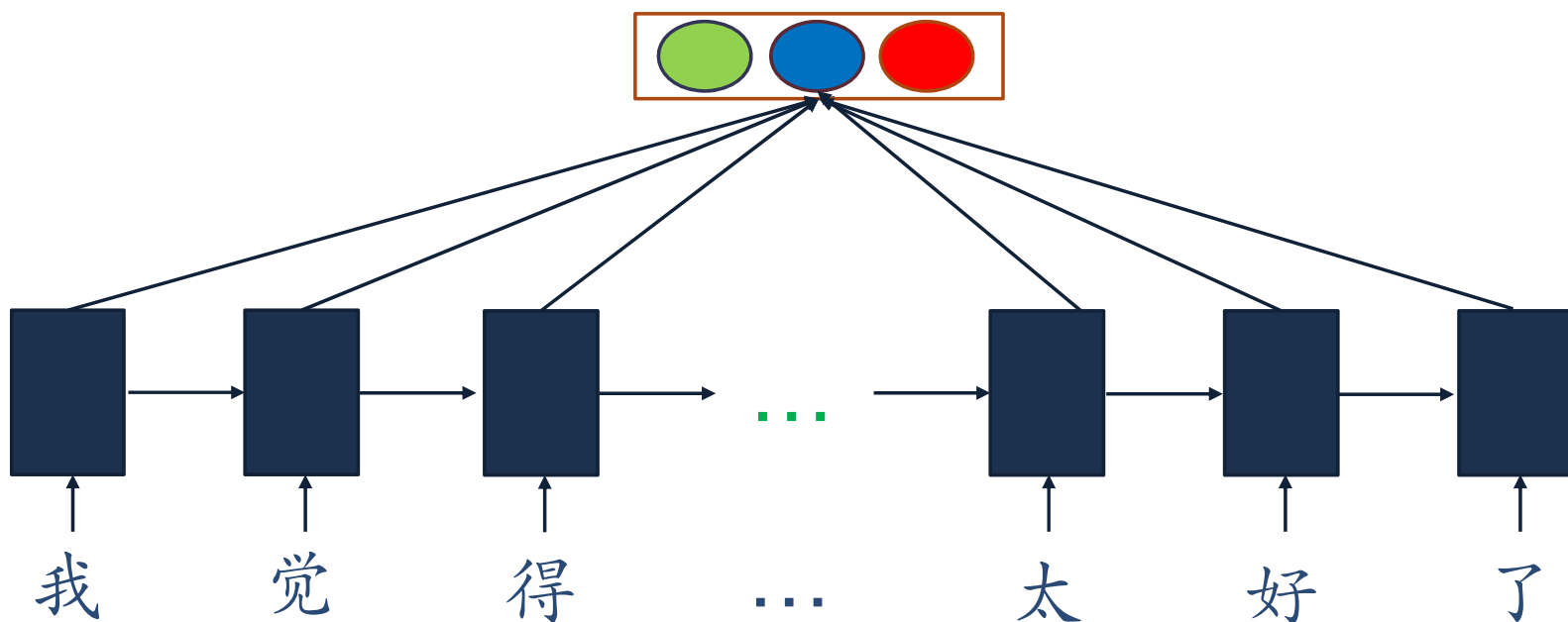
Positive (正面)

这部电影太糟了

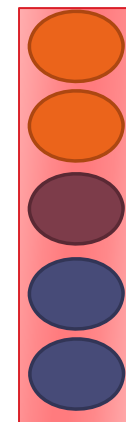
Negative (负面)

这部电影很棒

Positive (正面)

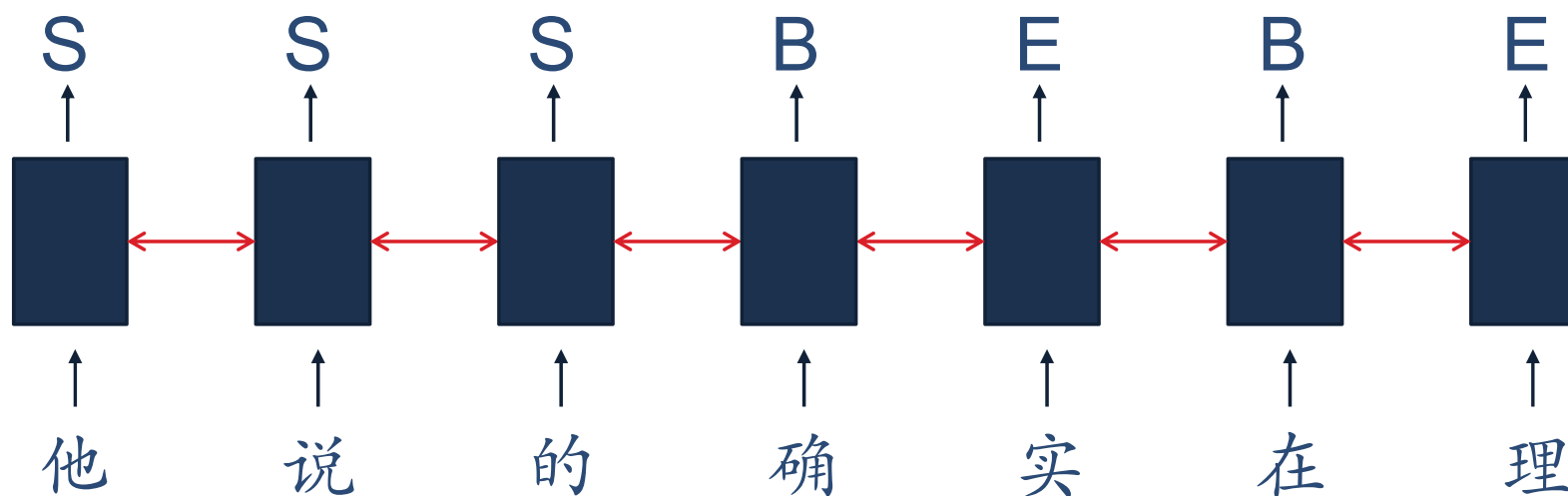


正面
中性
负面



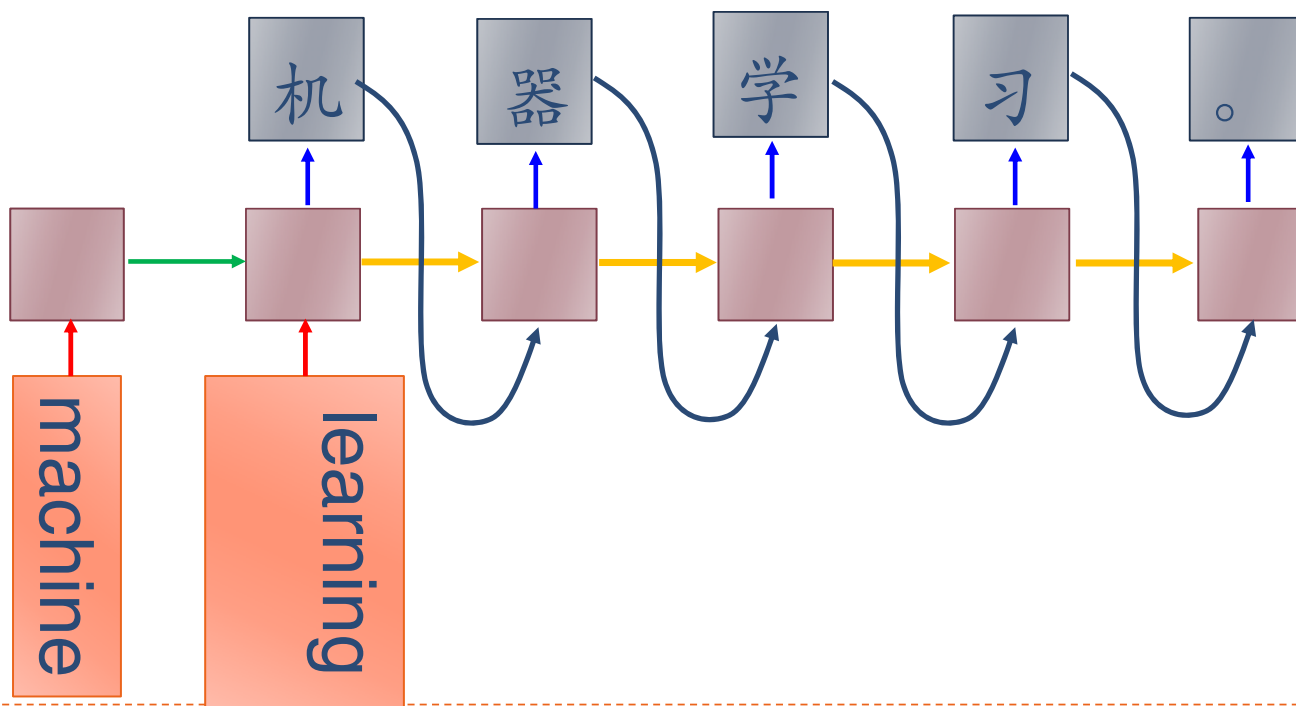
2 同步的序列到序列模式

► e.g. 中文分词



异步的序列到序列模式

► e.g. 机器翻译



1 随时间反向传播BPTT

▶ BPTT算法将RNN看做一个展开的多层前馈网络，其中“每一层”对应循环网络中的“每个时刻”。这样RNN可以按照前馈网络中的反向传播算法计算参数梯度。

▶ 在“展开”的前馈网络中，所有层的参数是共享的，因此，参数的真实梯度是所有“展开层”的参数梯度之和

▶ 计算偏导数 $\frac{\partial L}{\partial U}$

▶ 参数U和隐藏层在每个时刻k ($1 \leq k < t$) 的净输入 $\mathbf{z}_k = \mathbf{U} \mathbf{h}_{k-1} + \mathbf{W} \mathbf{x}_k + \mathbf{b}$ 有关，因此，第t时刻的损失函数 L_t 关于参数 U_{ij} 的梯度为

$$\frac{\partial L_t}{\partial u_{ij}} = \sum_{k=1}^t \frac{\partial^+ z_k}{\partial u_{ij}} \frac{\partial L_t}{\partial z_k} \quad \frac{\partial^+ z_k}{\partial u_{ij}} = \left[0, \dots, [\mathbf{h}_{k-1}]_j, \dots, 0 \right] \stackrel{\Delta}{=} \mathbf{I}_i \left([\mathbf{h}_{k-1}]_j \right)$$

1 随时间反向传播BPTT

- 定义误差项为第 t 时刻的损失对第 k 时刻隐藏神经层的净输入 $\mathbf{z}_k$ 的导数，则当 $1 \leq k < t$ 时

$$\begin{aligned}\delta_{t,k} &= \frac{\partial L_t}{\partial \mathbf{z}_k} \\ &= \frac{\partial \mathbf{h}_k}{\partial \mathbf{z}_k} \frac{\partial \mathbf{z}_{k+1}}{\partial \mathbf{h}_k} \frac{\partial L_t}{\partial \mathbf{z}_{k+1}} \\ &= \text{diag}(f'(\mathbf{z}_k)) \mathbf{U}^T \delta_{t,k+1}\end{aligned}$$

$$\frac{\partial L_t}{\partial \mathbf{u}_{ij}} = \sum_{k=1}^t [\delta_{t,k}]_i [\mathbf{h}_{k-1}]_j$$

- 将两个公式合并，得到

$$\frac{\partial L_t}{\partial \mathbf{U}} = \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

梯度消失/爆炸 (Vanishing Gradient Problem/Gradient Exploding problem)

► 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

► 其中

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\underbrace{\text{diag}(f'(\mathbf{z}_{\tau})) U^T}_{r} \right) \delta_{t,t}$$

由于梯度爆炸 ($r>1$) 或梯度消失 ($r<1$) 问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题 (Long-Term Dependencies)，即如果时刻 t 的输出 y_t 依赖于 k 时刻的输入 x_k ，当时间间隔 $t-k$ 比较大时，简单神经网络很难建立这种长距离的依赖关系。

长程依赖问题

▶ 循环神经网络在时间维度上非常深！

- ▶ 梯度消失或梯度爆炸

▶ 如何改进？

- ▶ 梯度爆炸问题

- ▶ 权重衰减
- ▶ 梯度截断

- ▶ 梯度消失问题

- ▶ 改进模型

长程依赖问题

► 梯度爆炸

- 权重衰减，即通过给参数增加l1或者l2范数的正则项来限制参数的取值范围，从而使得 $r \leq 1$.
- 梯度截断，一种启发式方法，当梯度的模大于一定阈值时，就将它截断为一个较小的数

► 梯度消失

$$\mathbf{h}_t = f(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{x}_t + \mathbf{b})$$

- 改变模型，循环边改为线性依赖关系，比如令 $\mathbf{U}=\mathbf{I}$, $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{I}$, 即

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta)$$

其中， $g()$ 是一个非线性函数

长程依赖问题

▶ 梯度消失

- ▶ 增加非线性 $\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$

这样 $\mathbf{h}_t$ 和 $\mathbf{h}_{t-1}$ 之间既有线性关系，也有非线性关系

▶ 接上述改进，依然存在的问题

- ▶ 梯度爆炸，即依据上式计算误差项时 $\delta_{t,k} = \text{diag}(f'(z_k)) \mathbf{U}^T \delta_{t,k+1}$ 出现梯度过大
- ▶ 记忆容量，即随着 $\mathbf{h}_t$ 的不断累积存储新的输入信息，会发生饱和现象 ($g()$ 为饱和型函数)

1 长短期记忆网络 LSTM

- ▶ 长短期记忆网络 (Long Short-Term Memory Network, LSTM) 在公式 $\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$ 的基础上, LSTM主要改进以下两个方面:

- ▶ 1) 新的内部状态, 即LSTM引入一个新的内部状态(internal state) $\mathbf{c}_t \in \mathbb{R}^D$ 专门进行线性的循环信息传递, 同时非线性的输出信息给隐藏层的外部状态 $\mathbf{h}_t \in \mathbb{R}^D$ 。内部状态 $\mathbf{c}_t$ 通过下面的公式计算:

$$\begin{aligned}\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)\end{aligned}$$

- ▶ 其中 $\mathbf{f}_t \in [0, 1]^D$, $\mathbf{i}_t \in [0, 1]^D$ 和 $\mathbf{o}_t \in [0, 1]^D$ 为三个门gate来控制信息传递的路径, $\odot$ 为向量元素相乘, $\mathbf{c}_{t-1}$ 为上一时刻的记忆单元

1 长短期记忆网络 LSTM

- ▶ $\tilde{\mathbf{c}}_t \in \mathbb{R}^D$ 是通过非线性函数得到的候选状态，即

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

- ▶ 在每个时刻 t , LSTM网络的内部状态 $\mathbf{c}_t$ 记录了到当前时刻为止的历史信息。
- ▶ 2) 门控机制，前述两个公式（如下）分别有输入门 $\mathbf{i}_t$ 、遗忘门 $\mathbf{f}_t$ 和输出门 $\mathbf{o}_t$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- ▶ 此三门的作用：
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- ✓ 遗忘门 $\mathbf{f}_t$ 控制上一时刻的内部状态 $\mathbf{c}_{t-1}$ 需要遗忘多少信息
- ✓ 输入门 $\mathbf{i}_t$ 控制当前时刻候选状态 $\tilde{\mathbf{c}}_t$ 有多少信息需要保存
- ✓ 输出门 $\mathbf{o}_t$ 控制当前时刻的内部状态 $\mathbf{c}_t$ 有多少信息需要输出给外部状态 $\mathbf{h}_t$

1 长短期记忆网络 LSTM

- ▶ LSTM网络中的“门”是一种“软”门，取值为（0,1）之间，即以一定的比例允许信息通过。三个门的计算方式：

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)$$

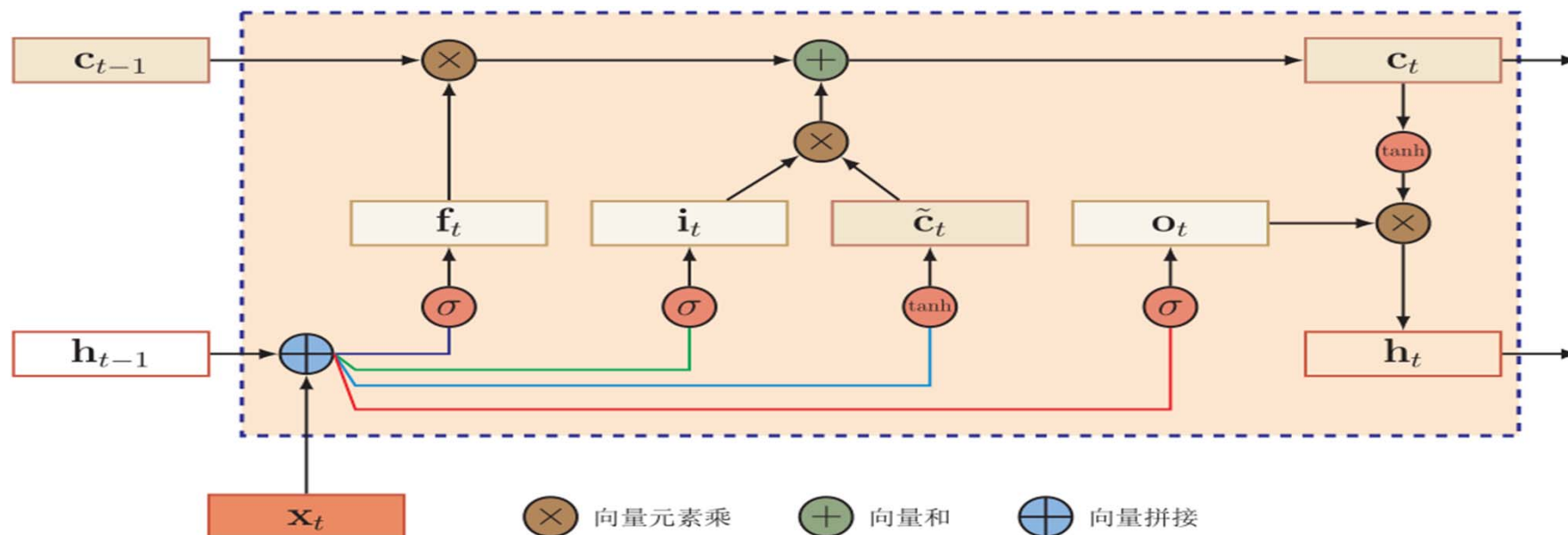
$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o)$$

其中 $\mathbf{x}_t$ 为当前时刻的输入， $\mathbf{h}_{t-1}$ 为上一时刻的外部状态

- ▶ 下图给出了LSTM的循环单元结构，其计算过程：
 - ✓ 1) 利用上一时刻的外部状态 $\mathbf{h}_{t-1}$ 和当前时刻的输入 $\mathbf{x}_t$ ，计算三个门及候选状态 $\tilde{\mathbf{c}}_t$ ；
 - ✓ 2) 结合三门来更新记忆单元 $\mathbf{c}_t$ ；
 - ✓ 3) 结合输出门 $\mathbf{o}_t$ ，将内部状态的信息传递给外部状态 $\mathbf{h}_t$

1 长短期记忆神经网络: LSTM



$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

3 门控循环单元网络 GRU gated recurrent unit

▶ GRU是一种比LSTM简单的循环神经网络

- ▶ 与LSTM不同，GRU不引入额外的记忆单元，GRU引入了更新门 (Update Gate)来控制当前状态需要从历史状态中保留多少信息，以及需要从候选状态中接受多少新信息

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$



$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot g(\mathbf{x}, \mathbf{h}_{t-1}; \theta)$$

$$\mathbf{z}_t \in [0, 1]^D \text{ 为更新门 } \mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z)$$

- ▶ 在GRU中， $g(\mathbf{x}, \mathbf{h}_{t-1}; \theta)$ 定义为

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

$\tilde{\mathbf{h}}_t$ 表示当前时刻的候选状态， $\mathbf{r}_t \in [0, 1]^D$ 为重置门 (Reset Gate)

3 门控循环单元网络 GRU

- 重置门(reset gate) $\mathbf{r}_t$ 的定义为

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r)$$

其用来控制候选状态 $\tilde{\mathbf{h}}_t$ 的计算是否依赖上一时刻的状态 $\mathbf{h}_{t-1}$

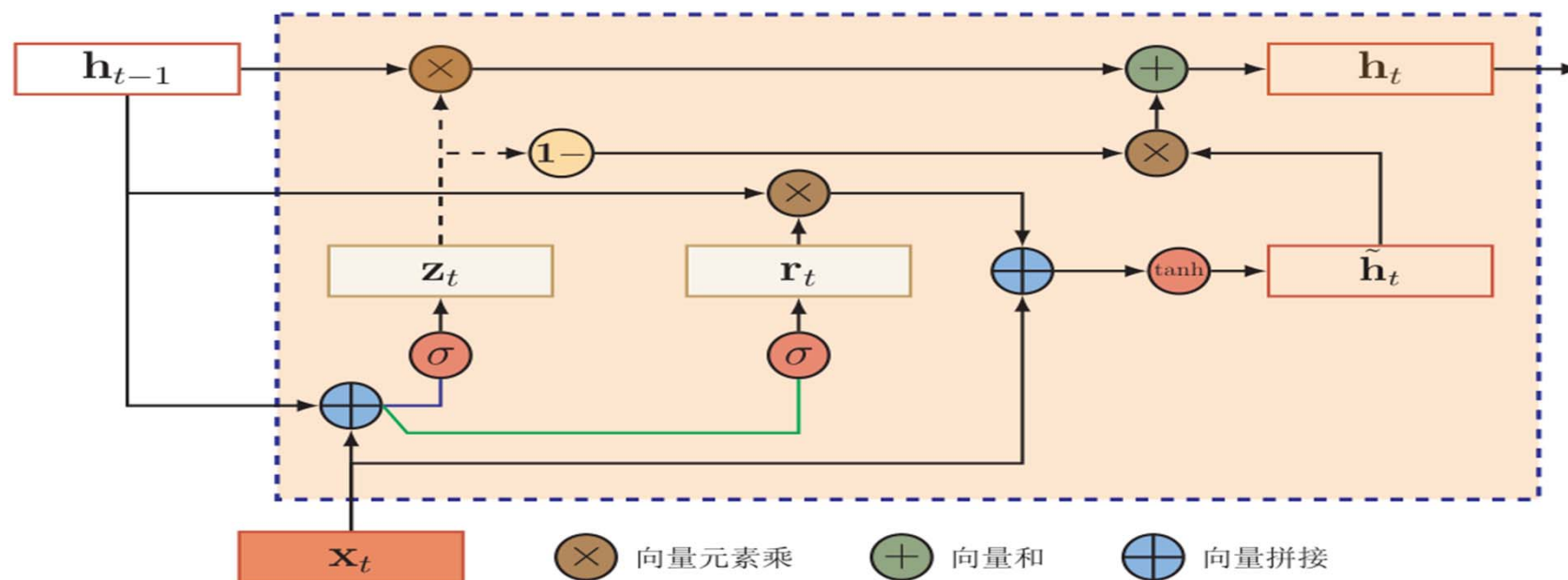
- 综上，GRU的状态更新方式为

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \mathbf{g}(\mathbf{x}, \mathbf{h}_{t-1}; \theta)$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

- 下图给出了GRU网络的循环单元结构

3 门控循环单元网络 GRU



重置门

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$$

更新门

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

《神经网络与深度学习》

网络优化与正则化

周晓飞
自动化学院



将神经网络应用到机器学习时，存在两大类难点：

► 优化问题（**训练**） optimization

- ▶ 损失函数是非凸函数
- ▶ 参数巨多，需要的数据量较大
- ▶ 存在梯度消失或爆炸

► 泛化问题（**测试**） generalization

- ▶ 复杂度高，易过拟合
- ▶ 需要正则化改进网络泛化能力

本章主要内容包括：

► 网络优化 optimization

优化算法

参数初始化

数据预处理

逐层归一化

超参数优化

► 网络正则化 regularization

l_1/l_2 正则化

权重衰减

提前停止

丢弃法

数据增强和标签平滑

1 Mini-Batch Gradient Descent

► Select K Samples $\mathcal{I}_t = \{\mathbf{x}^{(k)}, y^{(k)}\}_{k=1}^K$, compute partial derivative

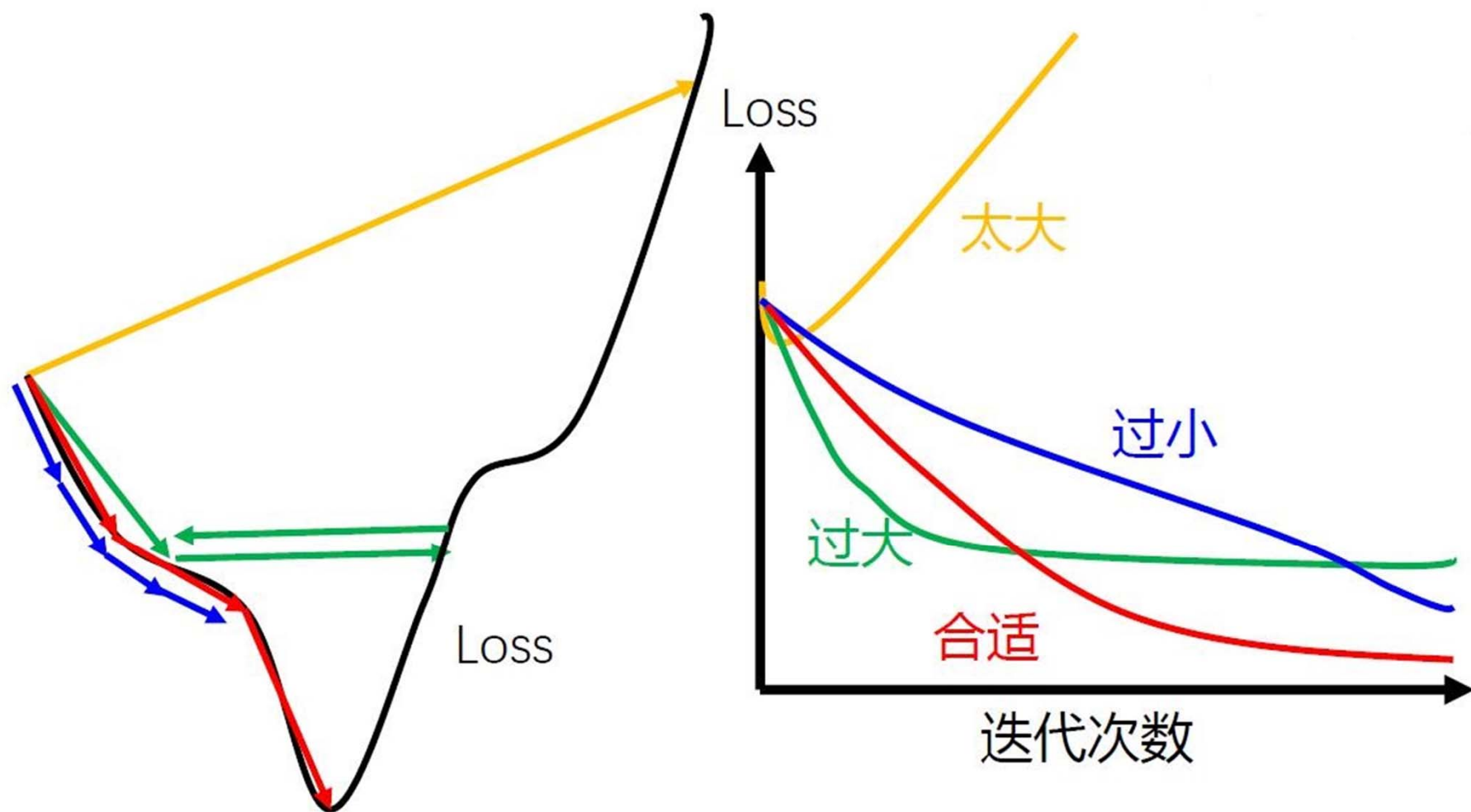
► Gradient: $\mathbf{g}_t(\theta) = \frac{1}{K} \sum_{(\mathbf{x}^{(k)}, y^{(k)}) \in \mathcal{I}_t} \frac{\partial \mathcal{L}(\mathbf{y}^{(k)}, f(\mathbf{x}^{(k)}, \theta))}{\partial \theta} \quad \mathbf{g}_t \triangleq \mathbf{g}_t(\theta_{t-1})$

► Update parameters $\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$

$\alpha > 0$ 为 learning rate

Some key points:

- K mini-batch
- $\mathbf{g}_t$ gradient estimation
- α learning rate



►神经网络常用优化方法的汇总

表 7.1 神经网络常用优化方法的汇总

类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

如何改进?

Reference:

1. [An overview of gradient descent optimization algorithms](#)
2. [Optimizing the Gradient Descent](#)

▶ 标准的（小批量）梯度下降

$$\Delta\theta_t = -\alpha \mathbf{g}_t$$

▶ 学习率

- ▶ 学习率衰减
- ▶ 学习率预热
- ▶ 周期性学习率
- ▶ 自适应: Adagrad、Adadelata、RMSprop

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

实际更新方向 梯度方向

▶ 梯度

- ▶ Momentum
 - ▶ 计算负梯度的“加权移动平均”作为参数的更新方向
- ▶ Nesterov accelerated gradient
- ▶ 梯度截断（抑制梯度爆炸）

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha \mathbf{g}_t$$

Adam

Adam is better choice!

▶ 综合

- ▶ Adam

▶ 预训练初始化 pre-trained initialization

- ▶ 不同参数初始值会收敛到不同的局部最优解
- ▶ 一个已在大规模数据集上训练过的模型可以提供一个好的参数初始值
- ▶ 预训练模型在目标任务上的学习过程称为精调Fine-tuning

▶ 随机初始化 Random initialization

- ▶ 对称权重 (0初始化)：参数为0，神经元输出相同，BP梯度相同，参数更新相同，参数相同
- ▶ 每个参数随机初始化，提高神经元之间的区分性
- ▶ 固定方差的参数初始化、方差缩放的参数初始化 Xavier & He、正交初始化

▶ 固定值初始化

- ▶ 经验固定值来初始化，如偏置通常用0来初始化

1 基于固定方差的参数初始化

▶ 从固定均值和方差的分布中采样来生成参数的初始值，有以下两种

▶ 1) Gaussian分布初始化 $N(0, \sigma^2)$

▶ Gaussian初始化方法是最简单的初始化方法，参数从一个固定均值（比如0）和固定方差（比如0.01）的Gaussian分布进行随机初始化。

▶ 2) 均匀分布初始化

▶ 参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

x 在 $[a, b]$ 内均匀分布，

$$\text{则 } \text{var}(x) = \frac{(b-a)^2}{12}$$



x 在 $[-r, r]$ 内均匀分布， $\text{var}(x) = \sigma^2$ ，

$$\text{则 } r = \sqrt{3\sigma^2}$$

关键在于如何设置方差：

参数取值范围过小，则神经元输出小，多层之后慢慢消失；亦会使sigmoid丢失非线性；

参数取值范围过大，易导致sigmoid激活值变得饱和

2 基于方差缩放的参数初始化

- ▶ 初始化网络时，为了缓解梯度消失或者梯度爆炸，要尽可能的保持每个神经元的输入和输出的方差一致，根据神经元连接数量来自适应地调整初始化分布的方差，这类方法称为 **方差缩放 (Variance Scaling)**

- ▶ **Xavier初始化**，由每层神经元数量自动计算初始化参数方差

- ▶ 1层神经元和l-1层神经元关系

$$\mathbf{a}^{(l)} = f\left(\sum_{i=1}^{M_{l-1}} w_i^{(l)} \mathbf{a}_i^{(l-1)}\right)$$

- ▶ 假设 $w_i^{(l)}$ 和 $\mathbf{a}_i^{(l-1)}$ 的均值为0，且相互独立，则 $\mathbf{a}^{(l)}$ 的均值、方差为

$$\mathbb{E}[\mathbf{a}^{(l)}] = \mathbb{E}\left[\sum_{i=1}^{M_{l-1}} w_i^{(l)} \mathbf{a}_i^{(l-1)}\right]$$

$$= \sum_{i=1}^{M_{l-1}} \mathbb{E}[w_i^{(l)}] \mathbb{E}[\mathbf{a}_i^{(l-1)}] = 0$$

$$\text{var}(\mathbf{a}^{(l)}) = \text{var}\left(\sum_{i=1}^{M_{l-1}} w_i^{(l)} \mathbf{a}_i^{(l-1)}\right)$$

$$= \sum_{i=1}^{M_{l-1}} \text{var}(w_i^{(l)}) \text{var}(\mathbf{a}_i^{(l-1)})$$

$$= M_{l-1} \text{var}(w_i^{(l)}) \text{var}(\mathbf{a}_i^{(l-1)})$$

2 基于方差缩放的参数初始化

- ▶ 由前述公式得到：输出信号方差被放大或缩小了 $M_{l-1} \text{var}(w_i^{(l)})$ ，为了保持输入输出方差一致，令 $\text{var}(w_i^{(l)}) = \frac{1}{M_{l-1}}$
- ▶ 类似，反向传播时，同样令 $\text{var}(w_i^{(l)}) = \frac{1}{M_l}$
- ▶ 折中，取 $\text{var}(w_i^{(l)}) = \frac{2}{M_l + M_{l-1}}$
- ▶ 若采用高斯分布初始化参数，则 $w_i^{(l)}$ 可按 $N\left(0, \frac{2}{M_l + M_{l-1}}\right)$ 初始化
- ▶ 若采用均匀分布初始化参数 $[-r, r]$ ，则 $r = \sqrt{\frac{6}{M_l + M_{l-1}}}$

这种根据每层神经元数量来自动计算初始化参数方差的方法称为**Xavier初始化**

2 基于方差缩放的参数初始化

► He初始化

- ▶ 当第1层神经元使用ReLU激活函数时，通常有一半的神经元输出为0，因此其前向输出对应权重方差变为 $\text{var}(w_i^{(l)}) = \frac{2}{M_{l-1}}$
- ▶ ReLU作为激活函数，采用高斯分布，则方差 $\frac{2}{M_{l-1}}$ ，若采用 $[-r, r]$ 均匀分布，则

$$r = \sqrt{6/M_{l-1}}$$

上述初始化方法就是He初始化方法

数据预处理

▶ 尺度不变性 (scale invariance)

- ▶ 如果一个机器学习算法在缩放全部或部分特征后不影响它的学习和预测，我们称算法具有 **尺度不变性**，线性分类器
- ▶ 对尺度敏感模型，须先对样本进行预处理，将样本特征各维度转换到同一尺度，并消除彼此之间的相关性 最近邻分类器
- ▶ 不同尺度的输入特征会增加神经网络的训练难度

▶ 影响梯度下降算法效率

- ▶ 不同尺度输入特征差异较大，梯度方向并不指向最优搜索方向

数据预处理

► 归一化 (Normalization)

- 泛指把数据特征转换为相同尺度的方法
- 假设有N个样本 $\{x^{(n)}\}_{n=1}^N$ ，对特征的每一维进行归一化

1) 缩放归一化，最大最小归一化 Min-max normalization

$$\hat{x}^{(n)} = \frac{x^{(n)} - \min_n(x^{(n)})}{\max_n(x^{(n)}) - \min_n(x^{(n)})}$$

2) 标准化 standardization 也叫Z值归一化 (Z-Score normalization)

$$\hat{x}^{(n)} = \frac{x^{(n)} - \mu}{\sigma} \quad \mu = \frac{1}{N} \sum_{n=1}^N x^{(n)}, \sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2$$

数据预处理

► 归一化 (Normalization)

3) 白化 whitening 用来降低数据之间的冗余性，减少了特征之间的相关性，且所有特征具有相同方差。常采用主成分分析PCA方法去掉各成分之间的相关性

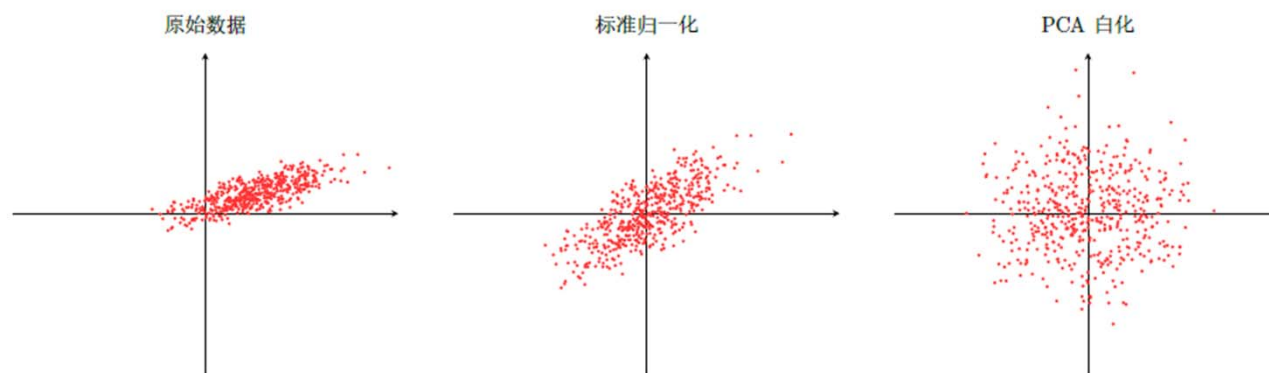


图 7.10 标准归一化和PCA 白化

逐层归一化

- ▶ 逐层归一化（**Layer-wise normalization**）可有效提升训练效率
 - ▶ 更好地尺度不变性
 - ▶ 如果一个神经层的输入分布发生了改变，那么其参数需要重新学习，此种现象称为**内部协变量偏移**（**Internal covariate shift**）。为此，需要对每层输入进行归一化，使得输入具有**尺度不变性**。
 - ▶ 更平滑的优化地形
 - ▶ 使得输入处于非饱和区，避免了梯度消失
 - ▶ 使得神经网络的**优化地形**（**Optimization landscape**）更加平滑

下面重点介绍：批量归一化、层归一化、权重归一化和局部响应归一化

批量归一化 Batch Normalization

▶ 批量归一化 Batch Normalization BN, 可对神经网络中任意的中间层进行归一化操作。

▶ 给定一个包含K个样本的小批量样本集合, 第l层神经元的净输入 $\mathbf{z}^{(1,1)}$, ..., $\mathbf{z}^{(K,1)}$ 计算均值和方差

$$\mu_B = \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,1)}, \sigma_B^2 = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,1)} - \mu_B) \odot (\mathbf{z}^{(k,1)} - \mu_B)$$

▶ 批量归一化(引入可学习参数缩放 γ 和平移 β , 远离线性区, 增加非线性)

$$\hat{\mathbf{z}}^{(1)} = \frac{\mathbf{z}^{(1)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \Rightarrow \quad \hat{\mathbf{z}}^{(1)} = \frac{\mathbf{z}^{(1)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \odot \gamma + \beta \stackrel{\Delta}{=} \text{BN}_{\gamma, \beta}(\mathbf{z}^{(1)})$$

▶ 作用于每层激活函数之前 $\mathbf{a}^{(1)} = f\left(\text{BN}_{\gamma, \beta}(\mathbf{z}^{(1)})\right) = f\left(\text{BN}_{\gamma, \beta}(\mathbf{W}\mathbf{a}^{(1-1)})\right)$

层归一化 Layer Normalization

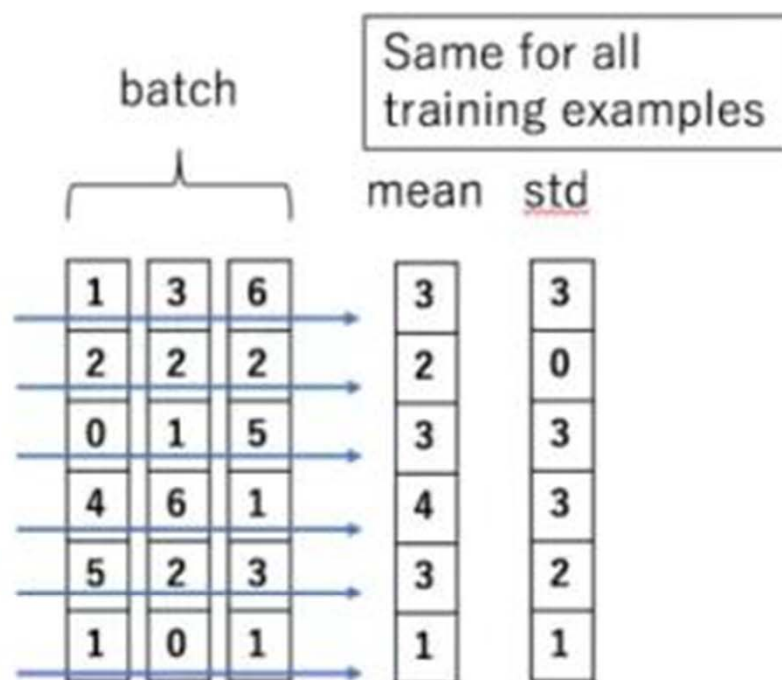
- ▶ 极小批量样本不适合BN，难以计算单个神经元的统计信息
- ▶ 对一个中间层所有神经元的归一化
 - ▶ 第1层M个神经元，净输入为 $\mathbf{z}^{(1)}$ ，则

$$\mu^{(1)} = \frac{1}{M_1} \sum_{i=1}^{M_1} z_i^{(1)}, \sigma^{(1)^2} = \frac{1}{M_1} \sum_{i=1}^{M_1} \left(z_i^{(1)} - \mu^{(1)} \right)^2$$

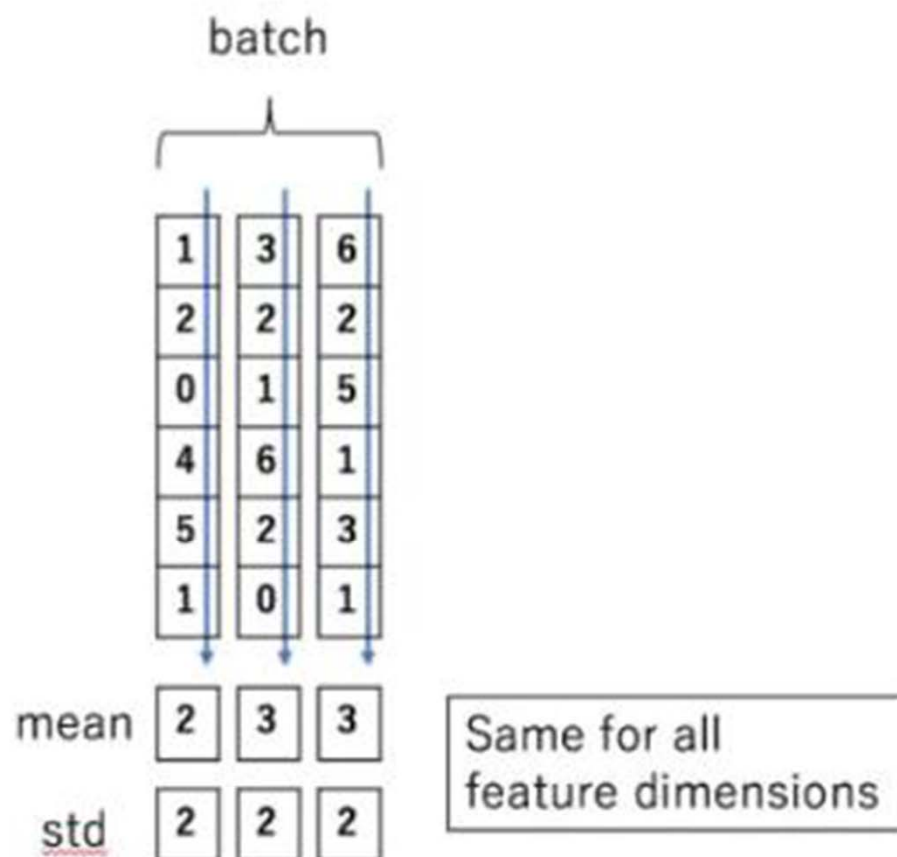
▶ 层归一化 $\hat{\mathbf{z}}^{(1)} = \frac{\mathbf{z}^{(1)} - \mu^{(1)}}{\sqrt{\sigma^{(1)^2} + \varepsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta} \stackrel{\Delta}{=} \text{LN}_{\gamma, \beta} \left(\mathbf{z}^{(1)} \right)$

- ▶ 可以应用在循环神经网络中，对循环神经层进行归一化操作，可有效缓解梯度消失或爆炸

Batch Normalization

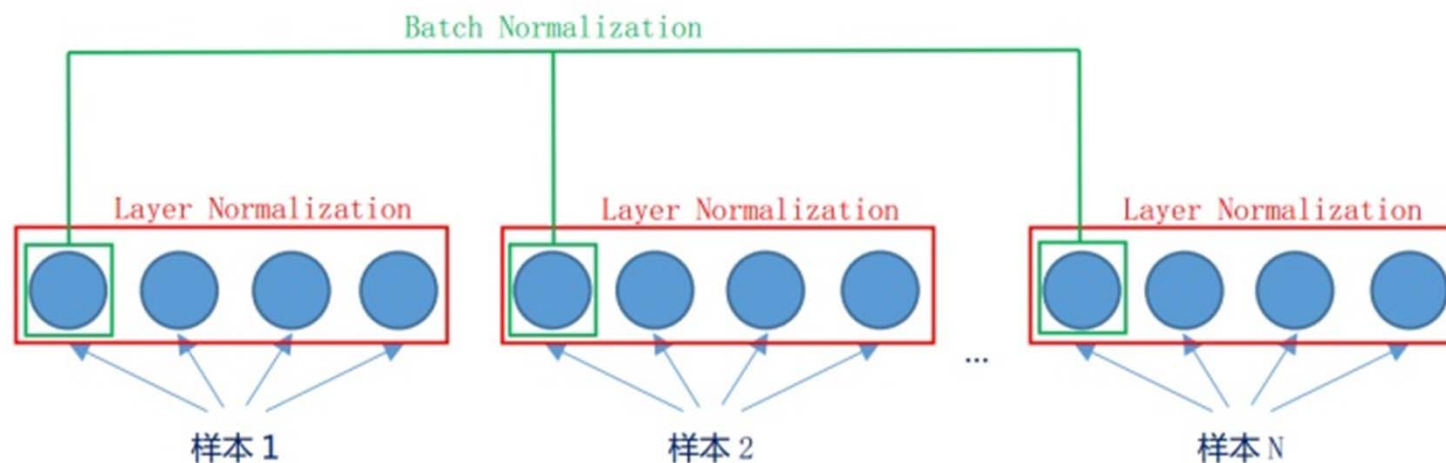


Layer Normalization



批量归一化VS层归一化

层归一化是对中间层的所有神经元节点进行归一化，均值方差均是标量
批量归一化则是对所有样本在中间层的归一化，均值方差是向量



超参数优化 Hyperparameter optimization

超参数对网络性能影响很大，常见的超参数有以下三类：

- ▶ **网络结构**，包括神经元之间的连接关系、层数、每层的神经元数量、激活函数的类型等
- ▶ **优化参数**，包括优化方法、学习率、小批量的样本数量等
- ▶ **正则化系数**

超参数优化主要存在两方面困难：

- ▶ 优化难，超参数优化是一个**组合优化**问题
- ▶ 配置难，**评估**超参数配置的时间代价高

超参数配置比较简单的方法：**网格搜索、随机搜索、贝叶斯优化、动态资源分配、神经架构搜索**

正则化

- ▶ 如何提高神经网络的泛化能力
 - ▶ ℓ_1 和 ℓ_2 正则化
 - ▶ 权重衰减
 - ▶ early stop
 - ▶ Dropout
 - ▶ 数据增强
 - ▶ 标签平滑

《神经网络与深度学习》

注意力机制与外部记忆

周晓飞
自动化学院



注意力模型

▶ 软性注意力机制 (soft attention mechanism)

▶ 注意力机制可以分为两步 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \in \mathbb{R}^{D \times N}$

◆ 1) 计算注意力分布 α ：每个输入和查询向量的相关性

▶ 引入和任务相关的查询向量 $\mathbf{q}$ ，以 $z \in [1, N]$ 表示被选择信息的索引位置 $z=n$ ，

▶ “软性”信息选择机制：在给定 $\mathbf{q}$ 和 $\mathbf{X}$ 的情况下，选择第 n 个输入向量的概率

$$\begin{aligned}\alpha_n &= p(z = n | \mathbf{X}, \mathbf{q}) \\ &= \text{soft max}(s(\mathbf{x}_n, \mathbf{q})) \\ &= \frac{\exp(s(\mathbf{x}_n, \mathbf{q}))}{\sum_{j=1}^N \exp(s(\mathbf{x}_j, \mathbf{q}))}\end{aligned}$$

▶ 其中， α_n 称为注意力分布， $s(\mathbf{x}, \mathbf{q})$ 为注意力打分函数

注意力模型

► 软性注意力机制 (soft attention mechanism)

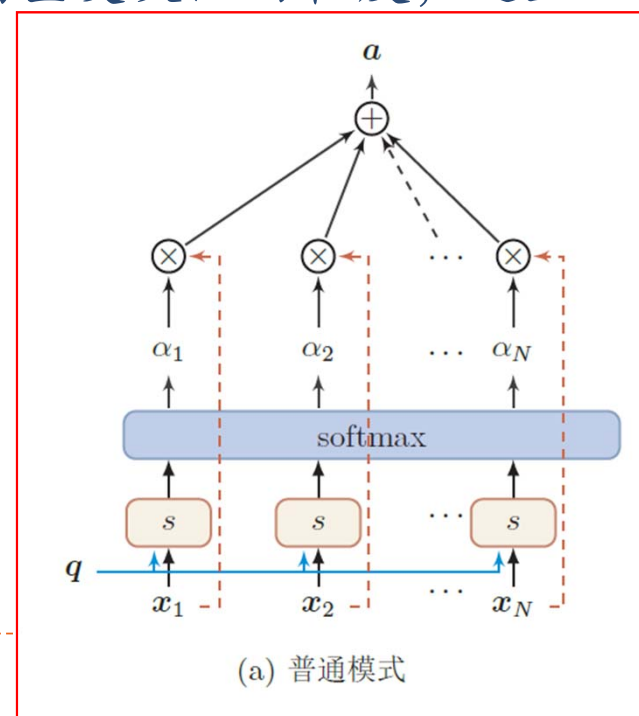
► 注意力机制可以分为两步 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \in \mathbb{R}^{D \times N}$

◆ 2) 加权平均

α_n 可以解释为给定任务相关的查询 q 时，第 n 个输入向量受关注的程度，这里采用“**软性**”信息选择机制对输入信息进行汇总，即

$$\text{att}(\mathbf{X}, \mathbf{q}) = \sum_{n=1}^N \alpha_n \mathbf{x}_n = \mathbb{E}_{z: p(z|\mathbf{X}, \mathbf{q})} [\mathbf{x}_z]$$

此式即为**软性注意力机制**。左图为例



注意力机制的变体

▶ 硬性注意力 (hard attention)

- ▶ 最大采样，即选取概率最大值对应的输入向量
- ▶ 随机采样，在注意力分布上随机采样
- ▶ 损失函数与注意力分布之间不可导!!!

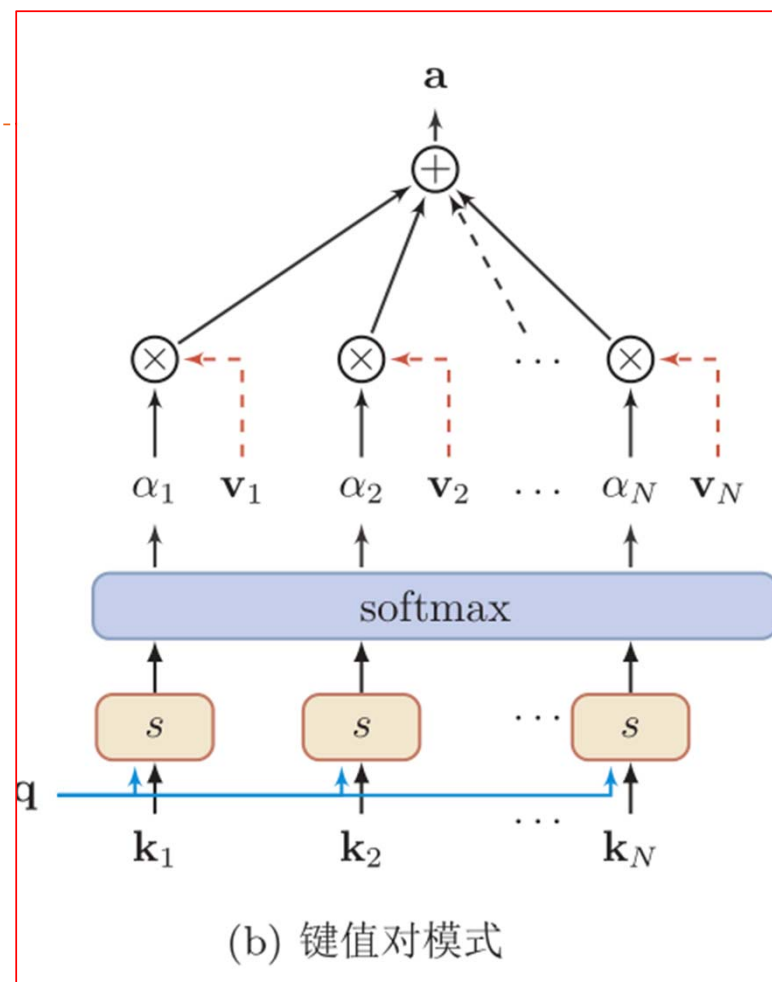
$$\text{att}(\mathbf{X}, \mathbf{q}) = \mathbf{x}_{\hat{n}} \quad \hat{n} = \arg \max_{n=1}^N \alpha_n$$

▶ 键值对注意力 (key-value pair attention)

- ▶ $(\mathbf{K}, \mathbf{N}) = [(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_N, \mathbf{v}_N)]$ 表示N个输入信息，给定查询量q时，注意力函数

$$\begin{aligned} \text{att}((\mathbf{K}, \mathbf{V}), \mathbf{q}) &= \sum_{n=1}^N \alpha_n \mathbf{v}_n \\ &= \sum_{n=1}^N \frac{\exp(s(\mathbf{k}_n, \mathbf{q}))}{\sum_j \exp(s(\mathbf{k}_j, \mathbf{q}))} \mathbf{v}_n \end{aligned}$$

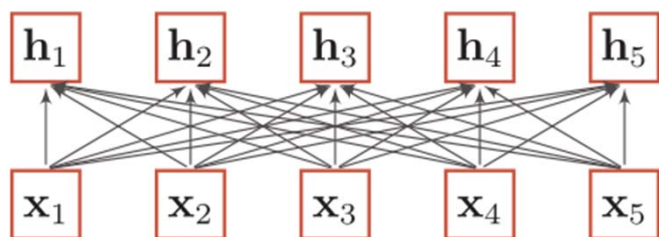
注意力机制的变体



自注意力模型 self-attention model

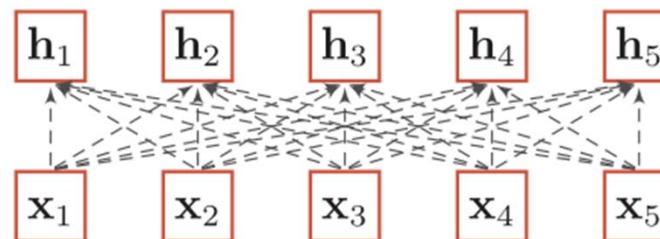
► 如何建立非局部 (Non-local) 的依赖关系

- 增加网络的层数
- 全连接 → 无法处理变长的输入序列；不同的输入长度，连接权重大小不同



(a) 全连接模型

无法处理变长问题



连接权重 α_{ij} 由注意力机制动态生成

自注意力模型 self-attention model

▶ 输入序列为 $X = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{D_x \times N}$

▶ 首先生成三个向量序列 **查询-键-值**

▶ 对每个输入 $\mathbf{x}_i$, 计算得到**查询向量** $\mathbf{q}_i$, **键向量** $\mathbf{k}_i$ 和**值向量** $\mathbf{v}_i$

$$\mathbf{Q} = \mathbf{W}_q \mathbf{X} \in \mathbb{R}^{D_k \times N} \quad \mathbf{K} = \mathbf{W}_k \mathbf{X} \in \mathbb{R}^{D_k \times N} \quad \mathbf{V} = \mathbf{W}_v \mathbf{X} \in \mathbb{R}^{D_v \times N}$$

$$\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_N] \quad \mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_N] \quad \mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_N]$$

▶ 计算 $\mathbf{h}_i$

$$\mathbf{h}_n = \text{att}((\mathbf{K}, \mathbf{V}), \mathbf{q}_n)$$

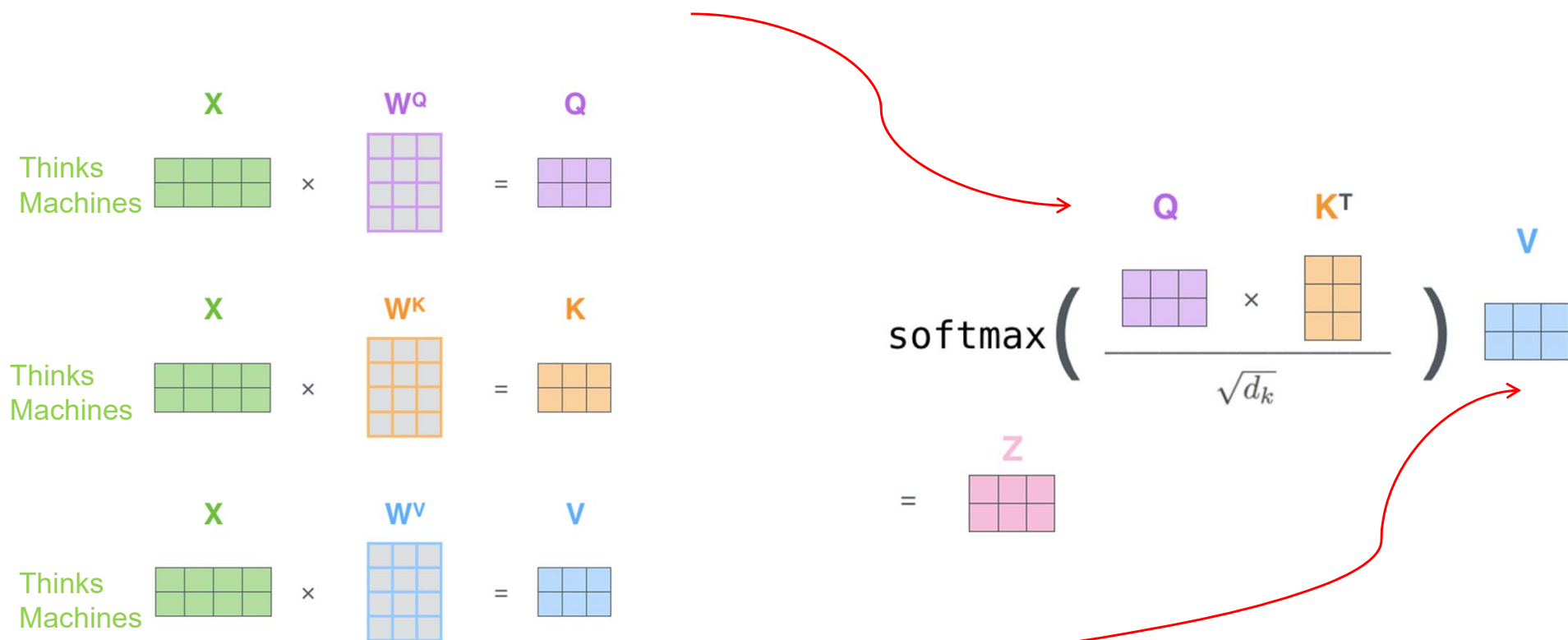
$$= \sum_{j=1}^N \alpha_{nj} \mathbf{v}_j$$

$$= \sum_{j=1}^N \text{soft max} \left(s(\mathbf{k}_j, \mathbf{q}_n) \right) \mathbf{v}_j$$

$$\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_N]$$

$$\mathbf{H} = \mathbf{V} \text{soft max} \left(\frac{\mathbf{K}^T \mathbf{Q}}{\sqrt{D_k}} \right)$$

QKV模式 (Query-Key-Value)



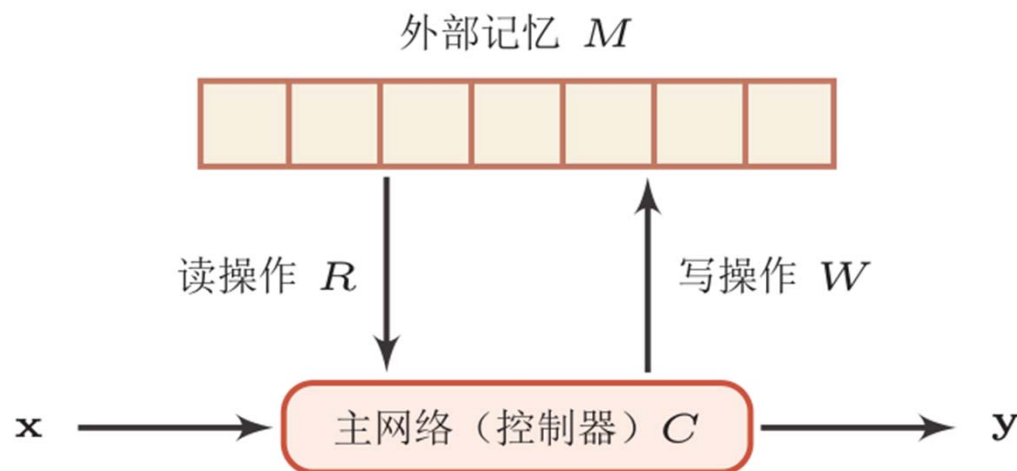
图片来源: <http://jalammar.github.io/illustrated-transformer/>

记忆增强网络

- ▶ 为了增强网络容量，引入辅助记忆单元，将一些信息保存在辅助记忆中，在需要时再进行读取，这样可以有效地增加网络容量。这个引入的辅助记忆单元一般称为**外部记忆**（**external memory**），以区别于循环神经网络的**内部记忆**（隐状态）
- ▶ 装备外部记忆的神经网络称为**记忆增强神经网络**（memory augmented neural network, MANN），简称**记忆网络**（memory network, MN）

记忆网络 (Memory Network)

- ▶ 记忆增强神经网络 (Memory Augmented Neural Network, MANN)
 - ▶ **主网络C** 控制器，负责信息处理和与外界交互（读写操作）
 - ▶ **外部记忆M** 存储信息，可分为很多记忆片段 $M=[m_1, m_2, \dots, m_N]$ ，以一定的结构进行组织
 - ▶ **读写操作 R** 根据主网络C生成的查询向量 q_w 和 q_r ，分别
 - 1) 用要写入的信息 a 来更新记忆模块 M ;
 - 2) 从外部记忆单元读取信息 r



记忆网络 (Memory Network)

- ▶ 记忆增强神经网络 (Memory Augmented Neural Network, MANN)
 - ▶ 这种结构化的外部记忆是带有地址的，即每个记忆片段都可以按地址读取和写入。
 - ▶ 要实现类似人脑神经网络的联想记忆能力，就需要按内容寻址的方式进行定位，然后进行读取或写入操作。
 - ▶ 按内容寻址通常是用注意力机制来进行。

外部记忆

▶ 外部记忆定义为矩阵 $M \in \mathbb{R}^{d \times k}$

- ▶ k 是记忆片段的数量, d 是每个记忆片段的大小
- ▶ 计算一个在所有记忆片段上的分布,
- ▶ 按内容寻址的方式进行定位, 然后读写

▶ 如何读? **注意力机制**

- ▶ 按内容寻址, 常用注意力机制 以读模块R为例

寻址 $\alpha_n = \text{soft max} \left(s(\mathbf{m}_n, \mathbf{q}_r) \right)$

内容
读取 $\mathbf{r} = \sum_{n=1}^N \alpha_n \mathbf{m}_n$

外部记忆

- ▶ 记忆增强神经网络 (Memory Augmented Neural Network, MANN)
 - ▶ 计算注意力分布的过程相当于计算机的“寻址”过程
 - ▶ 信息加权求和的过程相当于计算机的“内容读取”过程故而，结构化的外部记忆也是一种联想记忆（基于内容寻址的存储）
- ▶ 通过引入外部记忆，将神经网络的参数和记忆容量“分离”
即在少量网络参数的条件下，大幅增加网络容量
- ▶ 注意力机制可看作一个接口，将信息的存储与计算分离

典型记忆网络-端到端记忆网络

▶ 端到端记忆网络 End-To-End Memory Network, MemN2N

- ▶ 可微的网络结构，能够多次从外部记忆中读取信息
- ▶ 给定待存储信息 $m_{1:N} = \{m_1, \dots, m_N\}$ ，转换为两组记忆片段（键值对）**A寻址，C输出**

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_N] \quad \mathbf{C} = [\mathbf{c}_1, \dots, \mathbf{c}_N]$$

- ▶ 主网络根据输入 $\mathbf{x}$ 得到查询向量 $\mathbf{q}$,使用**键值对注意力机制**来从外部记忆读取相关信息 $\mathbf{r}$,并产生输出 $\mathbf{y}$

$$\mathbf{r} = \sum_{n=1}^N \text{soft max} \left(\mathbf{a}_n^T \mathbf{q} \right) \mathbf{c}_n$$

$$\mathbf{y} = f(\mathbf{q} + \mathbf{r})$$

$f(\cdot)$ 为预测函数

多跳操作 Multi-Hop

- ▶ 为了实现更复杂的计算，让主网络和外部记忆进行多轮交互
 - ▶ 第 $k-1$ 次的结果得到第 k 次的输出，即新的查询向量

$$\mathbf{q}^{(k)} = \mathbf{q}^{(k-1)} + \mathbf{r}^{(k-1)}$$

- ▶ 第 k 轮交互的外部记忆 $A(k)$ 和 $C(k)$ ，主网络从外部记忆读取信息（只读）为

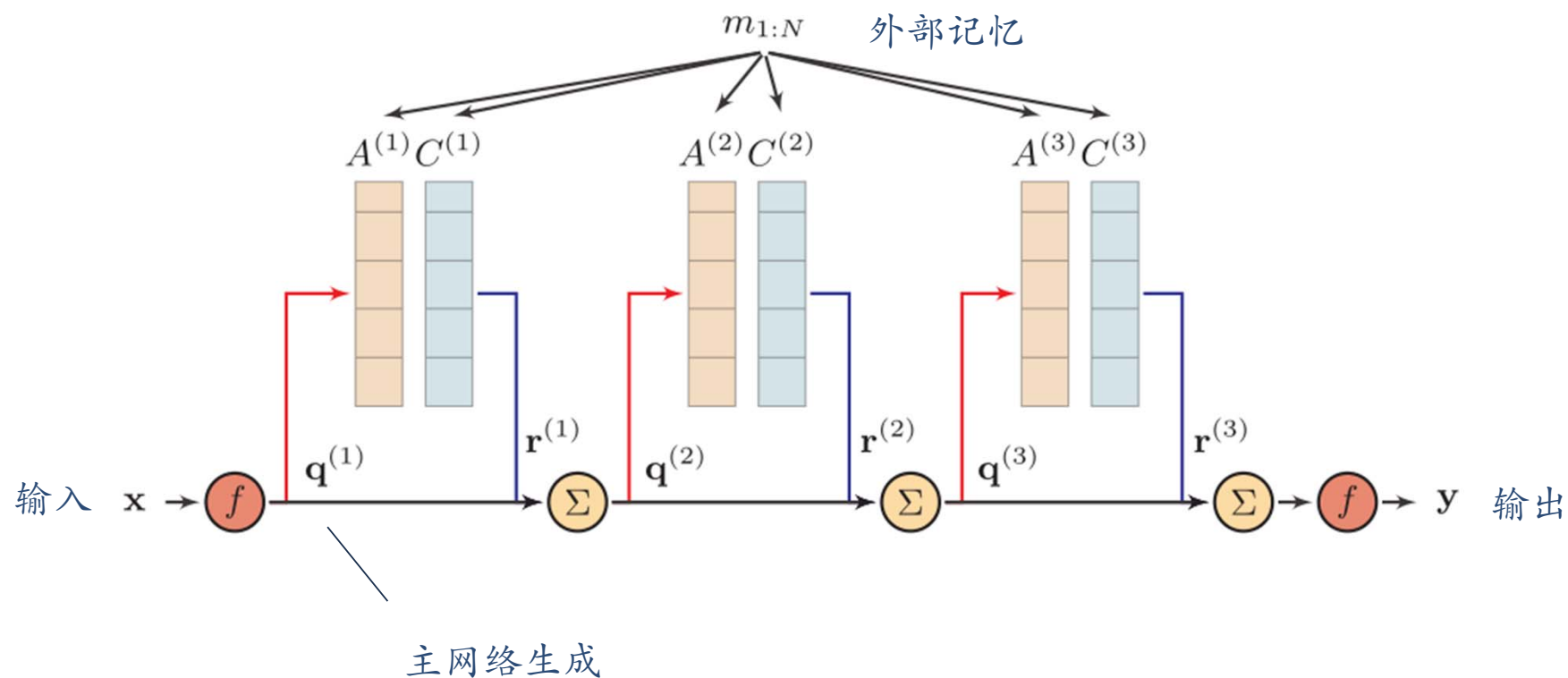
$$\mathbf{r}^{(k)} = \sum_{n=1}^N \text{soft max} \left(\left(\mathbf{a}_n^{(k)} \right)^T \mathbf{q}^{(k)} \right) \mathbf{c}_n^{(k)}$$

- ▶ K 轮交互后，用 $\mathbf{y} = \mathbf{f}(\mathbf{q}^{(K)} + \mathbf{r}^{(K)})$ 进行预测

- ▶ 这种多轮的交互方式也被称为多跳（参数共享、多轮外部记忆共享）

端到端记忆网络

Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2015). End-To-End Memory Networks, 1–11. <http://arxiv.org/abs/1503.08895>

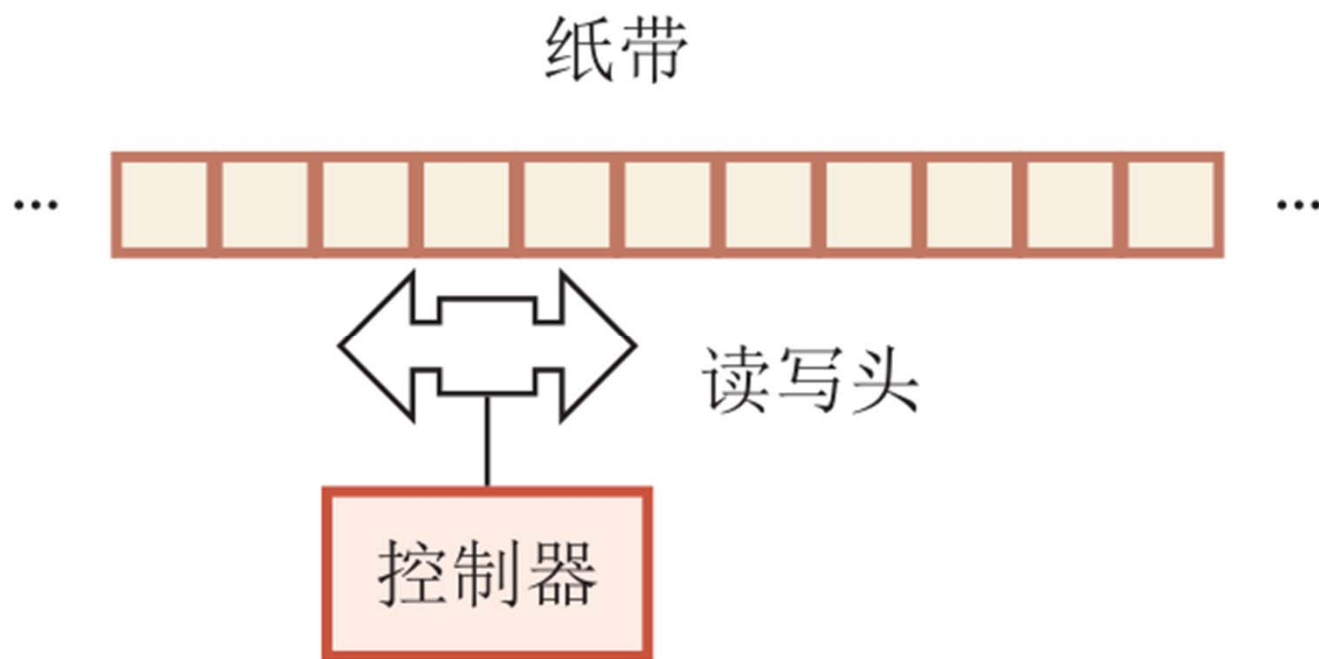


典型记忆网络-神经图灵机

▶ 图灵机 Turing Machine

- ▶ 图灵于1936年提出的一种抽象数学模型，可以用来模拟任何可计算问题。
- ▶ 构成：
 - ✓ 无限长纸带：存储符号
 - ✓ 符号表：纸带上符号的集合
 - ✓ 读写头：指向纸带上的方格的指针，可读取、擦除、写入当前方格中的内容
 - ✓ 状态寄存器：保存图灵机当前所处的状态，包括两个特殊的状态（起始状态、终止状态）
 - ✓ 控制规则：根据当前状态及当前读写头的符号来确定写头下一步的动作，令机器进入一个新的状态

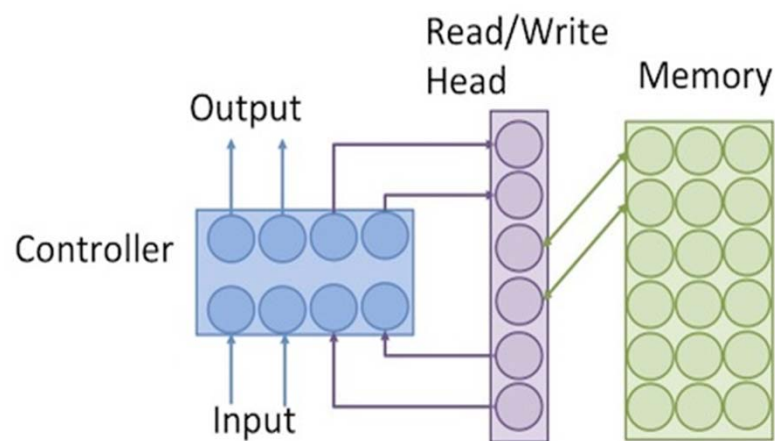
典型记忆网络-神经图灵机



神经图灵机 Neural Turing Machine, NTM

► 组件

- 控制器 前馈 or 循环神经网络
- 外部记忆 $\mathbf{M} \in \mathbb{R}^{D \times N}$ N个记忆片段 每个D维
- 外部记忆可以进行读写操作



图片来源: <http://cpmarkchang.logdown.com/posts/279710-neural-network-neural-turing-machine>

Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. Arxiv, 1–26. <http://arxiv.org/abs/1410.5401>

神经图灵机示例

► t 时刻，外部记忆 $\mathbf{M}_t = [\mathbf{m}_{t,1}, \dots, \mathbf{m}_{t,N}]$

► **读操作** 从 $\mathbf{M}_t$ 读取信息 $\mathbf{r}$ （读向量）

$$\alpha_{t,n} = \text{soft max} \left(s(\mathbf{m}_{t,n}, \mathbf{q}_t) \right)$$

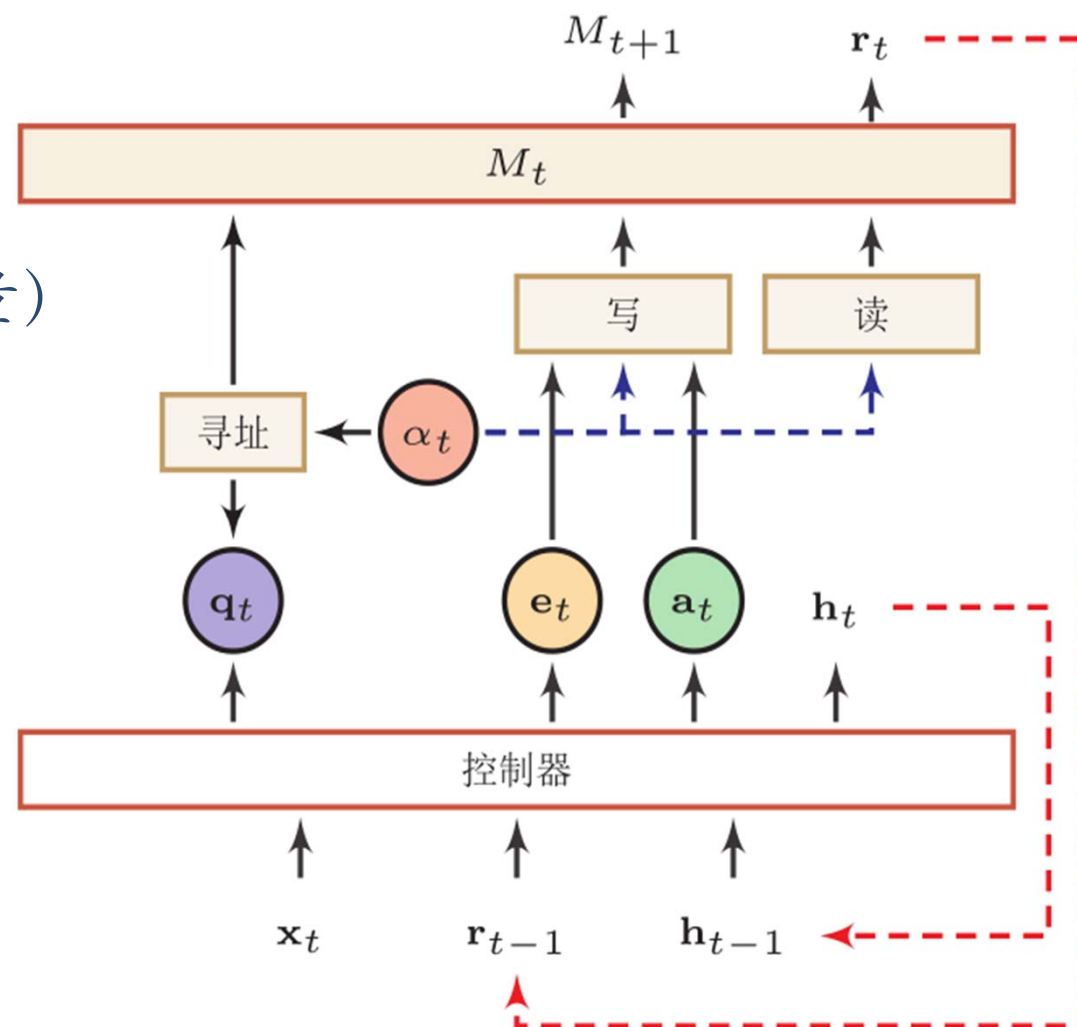
$$\mathbf{r}_t = \sum_{n=1}^N \alpha_{t,n} \mathbf{m}_{t,n}$$

► **写操作**

► 删除向量 $\mathbf{e}_t$ 和增加向量 $\mathbf{a}_t$

$$\mathbf{m}_{t+1,n} = \mathbf{m}_{t,n} \left(1 - \alpha_{t,n} \mathbf{e}_t \right) + \alpha_{t,n} \mathbf{a}_t$$

► 得到下一时刻的外部记忆 $\mathbf{M}_{t+1}$



Hopfield网络

► 神经网络作为一种记忆的存储和检索模型。

- 一种循环神经网络模型
- 所有神经元互相连接（自身不连接）

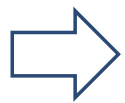
► 更新规则（m个神经元，第i个神经元）

$$s_i = \begin{cases} +1 & \sum_{j=1}^M w_{ij}s_j + b_i \geq 0 \\ -1 & \text{else} \end{cases}$$

$$w_{ii} = 0 \quad \forall i \in [1, M]$$

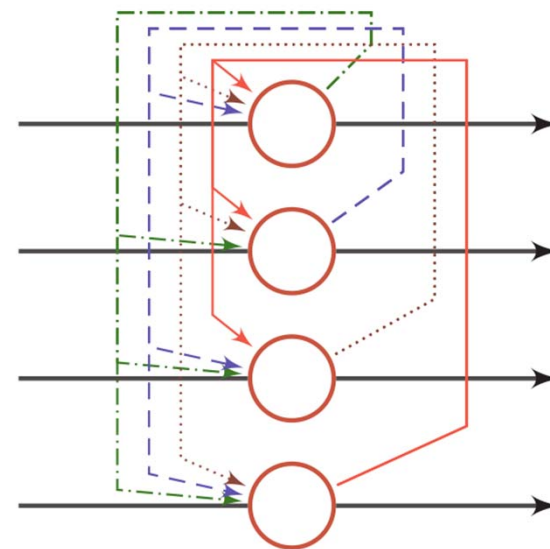
$$w_{ij} = w_{ji} \quad \forall i, j \in [1, M]$$

同步更新



$$\mathbf{s}_t = f(\mathbf{W}\mathbf{s}_{t-1} + \mathbf{b})$$

- 更新：同步和异步



《神经网络与深度学习》

无监督学习

周晓飞
自动化学院



无监督学习 (Unsupervised Learning)

- ▶ 无监督学习是指从无标签的数据中学习出一些有用的模式。
 - ▶ 一般直接从原始数据中学习 (无需人工标签或反馈等指导信息)
- ▶ 监督学习
 - ▶ 建立映射关系 $f: x \rightarrow y$, 寻找输入-输出之间的映射关系
- ▶ 无监督学习分类
 - ▶ 无监督特征学习 Unsupervised feature learning
 - ▶ 从无标签的训练数据中挖掘有效的特征或表示, 常用于降维、数据可视化、数据预处理
 - ▶ 概率密度估计 $p(x)$ 密度估计 Probabilistic density estimation
 - ▶ 根据一组训练数据来估计样本空间的概率密度
 - ▶ 参数密度估计和非参数密度估计
 - ▶ 聚类 Clustering
 - ▶ 根据一定的规则将一组数据分到不同的簇, 组内相似性高于组间相似性, K-mean

密度估计

- ▶ **概率密度估计** (Probabilistic Density Estimation) 简称**密度估计** (Density Estimation) 是基于一些观测样本来估计一个随机变量的概率密度函数。密度估计在数据建模、机器学习中使用广泛
- ▶ 密度估计方法可以分为两类
 - ▶ 参数密度估计 (Parametric Density Estimation) 是根据先验知识假设随机变量**服从某种分布**，然后通过训练样本来估计分布的参数。
 - ▶ 非参数密度估计 (Nonparametric Density Estimation) 是不假设数据服从某种分布，通过将样本空间划分为不同的区域并**估计每个区域的概率来近似数据的概率密度函数**。

参数密度估计

- ▶ 参数密度估计 (Parametric Density Estimation) 是根据先验知识假设随机变量服从某种分布, 然后通过训练样本来估计分布的参数
- ▶ 令 $D = \{\mathbf{x}^{(n)}\}_{i=1}^N$ 为从某个未知分布中独立抽取的 N 个训练样本, 假设这些样本服从一个概率分布函数 $p(\mathbf{x}; \theta)$, 其对数似然函数为

$$\log p(D; \theta) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)}; \theta)$$

- ▶ 如何计算 θ ?

- ▶ 最大似然估计 Maximum likelihood estimation, MLE

$$\theta^{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(\mathbf{x}^{(n)}; \theta)$$

- ▶ 根据不同的分布, 得出其对数似然函数, 令其参数导数为0即可得到

参数密度估计一般存在以下问题

▶ 模型选择问题

- ▶ 如何选择数据分布的密度函数？实际数据的分布往往是非常复杂的，而不是简单的正态分布或多项分布。

▶ 不可观测变量问题

- ▶ 即我们用来训练的样本只包含部分的可观测变量，还有一些非常关键的变量是无法观测的，这导致我们很难准确估计数据的真实分布。

▶ 维度灾难问题

- ▶ 高维数据的参数估计十分困难。随着维度的增加，估计参数所需要的样本数量指数增加。在样本不足时会出现过拟合。

非参密度估计

- ▶ 非参数密度估计 (Nonparametric Density Estimation) 是不假设数据服从某种分布，通过将样本空间划分为不同的区域并估计每个区域的概率来近似数据的概率密度函数。

非参密度估计

- ▶ 对于高维空间中的一个随机向量 $\mathbf{x}$ ，假设其服从一个未知分布 $p(\mathbf{x})$ ，则 $\mathbf{x}$ 落入空间中的小区域 $\mathcal{R}$ 的概率为

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}$$

- ▶ 给定 N 个训练样本 $D = \{\mathbf{x}^{(n)}\}_{i=1}^N$ ，落入区域 $\mathcal{R}$ 的样本数量 K 服从二项分布

$$P_K = \binom{N}{K} P^K (1 - P)^{1-K}$$

非参密度估计

- ▶ 当N 非常大时，我们可以近似认为

$$P \approx \frac{K}{N}$$

- ▶ 假设区域R足够小，其内部的概率密度是相同的，则有

$$P \approx p(\mathbf{x})V,$$

V为区域R的体积

- ▶ 结合上述两个公式，得到

$$p(\mathbf{x}) \approx \frac{K}{NV}$$

非参密度估计

- ▶ 根据前述公式，要准确地估计 $p(x)$ ，需要尽量使得样本数量 N 足够大，区域体积 V 尽可能地小。
- ▶ 在具体应用中，样本数量一般是有限的，过小的区域会导致落入该区域的样本比较少，这样估计的概率密度就不太准确
- ▶ 因此，实践中非参数密度估计通常使用两种方式：
 - ✓ (1) 固定区域大小 V ，统计落入不同区域的数量，这种方式包括直方图方法和核方法两种；
 - ✓ (2) 改变区域大小以使得落入每个区域的样本数量为 K ，这种方式称为K近邻方法。

《神经网络与深度学习》

模型独立的学习方式



周晓飞
自动化学院

模型独立的学习方式

- ▶ 之前介绍的很多模型或者网络都需要藉由训练数据获得参数，这导致每个任务都需要大量的训练数据：训练数据和真实数据分布一致，不同任务模型需要从零开始学习。
- ▶ 在实际应用中，我们往往很多时候是训练任务和目标任务的数据分布不一致，训练数据过少等，这时候机器学习的任务就会受到很大的局限。
- ▶ 很多时候我们需要一个模型可以很快的适应当前任务。为此，人们开始关注一些**新的学习方式**

模型独立的学习方式

▶ 本章介绍一些“模型独立的学习方式”

- ▶ E.g. 集成学习、协同学习、自训练、多任务学习、迁移学习、终身学习、小样本学习、元学习等。

▶ “模型独立”：这些学习方式不限于具体的模型

- ▶ E.g. 前馈神经网络、循环神经网络、其他模型

▶ 一种学习方式往往会对符合某种特性的模型更加青睐

- ▶ E.g. 集成学习往往和方差大的模型组合时效果显著

集成学习

▶ 三个臭皮匠赛过诸葛亮

定理 10.1: 对于 M 个不同的模型 $f_1(\mathbf{x}), \dots, f_M(\mathbf{x})$, 平均期望错误为 $\bar{\mathcal{R}}(f)$ 。基于简单投票机制的集成模型 $f^{(c)}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x})$, 其期望错误在 $\frac{1}{M} \bar{\mathcal{R}}(f)$ 和 $\bar{\mathcal{R}}(f)$ 之间。

- ▶ **集成学习** (Ensemble Learning) 就是通过某种策略将多个模型集成起来, 通过群体决策来提高决策准确率。
- ▶ 集成学习**首要的问题**是如何集成多个模型。比较常用的集成策略有直接平均、加权平均等。
- ▶ **最直接**的集成学习策略就是直接平均, 即“投票” $F(x) = \frac{1}{M} \sum_{m=1}^M f_m(x)$ 。

集成方式

- ▶ Bagging类（自举策略） Bagging 类方法是通过随机构造训练样本、随机选择特征等方法来提高每个基模型的独立性，代表性方法有Bagging和随机森林等
- ▶ Bagging（Bootstrap Aggregating）是一个通过不同模型的训练数据集的独立性来提高不同模型之间的独立性。我们在原始训练集上进行有放回的随机采样，得到M个比较小的训练集并训练M个模型，然后通过投票的方法进行模型集成。
- ▶ 随机森林（Random Forest）是在Bagging的基础上再引入了随机特征，进一步提高每个基模型之间的独立性。在随机森林中，每个基模型都是一棵决策树。

-
- ▶ 利用大量的无标注数据来提高监督学习的效果具有十分重要的意义，这种利用少量标注数据和大量无标注数据进行学习的方式称为**半监督学习**
 - ▶ 本节介绍两种半监督学习算法：**自训练和协同训练**

自训练 (Self-Training or Self-Teaching)

- ▶ **自训练**亦称作**自举法Bootstrapping**，具体思路如下：
 - ▶ 1) 使用标注数据来训练一个模型
 - ▶ 2) 使用这个模型来预测无标注样本的标签
 - ▶ 3) 把预测置信度较高的样本及其预测的伪标签加入训练集
 - ▶ 4) 重新训练新的模型，并不断重复这个过程

协同训练 (Co-Training)

- ▶ 协同训练对自训练的改进，通过两个基于不同视角的分类器来互相促进，协同训练就是利用数据的互补性进行自训练
- ▶ 不同数据具有不同视角
 - ▶ 条件独立（独立概率分布）
 - ▶ 充足和冗余性（单独可以训练一个正确的分类器）

协同训练 (Co-Training)

► Multi-View Learning

- 1) 根据不同视角分别训练两个模型 f_1 & f_2
- 2) 用 f_1 & f_2 在无标签数据集上进行预测
- 3) 各自 选取预测置信度高的样本加入训练集
- 4) 以此重新训练两个不同视角的模型，并不断重复

多任务学习 Multi-task Learning

- ▶ 一般的机器学习模型都是针对单一的特定任务，比如手写体数字识别、物体检测等。
- ▶ 不同任务的模型都是在各自的训练集上单独学习得到的。如果有两个任务比较相关，它们之间会存在一定的共享知识，这些知识对两个任务都会有所帮助。
- ▶ 这些共享的知识可以是表示（特征）、模型参数或学习算法等。目前，主流的多任务学习方法主要关注于表示层面的共享。

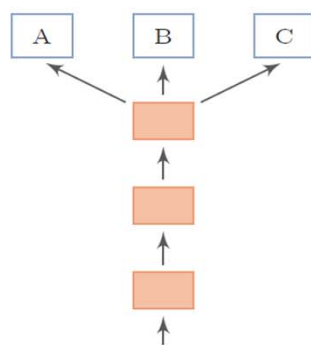
多任务学习 (Multitask Learning)

▶ 多任务学习

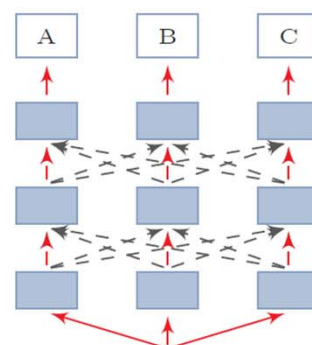
- ▶ 其指同时学习多个相关任务，实现知识共享，利用彼此之间的相关性来改进模型的性能和泛化能力
- ▶ 可视为归纳迁移学习 (Inductive Transfer Learning) ，利用包含在相关任务中的信息作为归纳偏置 (Inductive Bias) 来提高泛化能力
- ▶ 挑战在于如何设计多任务之间的共享机制

▶ 共享机制

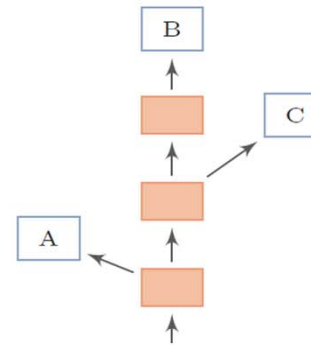
- ▶ 1) 硬共享模式
- ▶ 2) 软共享模式
- ▶ 3) 层次共享模式
- ▶ 4) 共享-私有模式



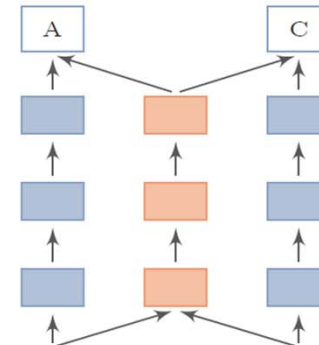
(a) 硬共享模式



(b) 软共享模式



(c) 层次共享模式



(d) 共享-私有模式

多任务学习 (Multitask Learning)

- ▶ 硬共享：低层特征共享，高层提取任务特定的特征
- ▶ 软共享：隐式共享，直接复制其它任务的隐状态或选用注意力机制从其他任务中获取信息
- ▶ 层次共享：多任务学习之任务也有高低之分，低级任务低层输出，高级任务高层输出
- ▶ 共享-私有：共享模块捕捉跨任务的共享特征，私有模块捕捉和特定任务相关的特征

迁移学习 Transfer Learning

- ▶ 若有一个**相关任务**已经有了大量的训练数据，虽然这些训练数据的分布和目标任务不同，但是由于**训练数据的规模比较大**，我们假设可以从中学学习某些可以泛化的知识，那么这些知识对目标任务会有一定的帮助
- ▶ 如何将相关任务的训练数据中的可泛化知识迁移到目标任务上，就是**迁移学习 (Transfer Learning)** 要解决的问题。

迁移学习 (Transfer Learning)

▶ 领域Domain $D = (X, Y, p(x, y))$

- ▶ 一个样本空间及其分布可以被称为一个领域 domain
- ▶ 给定两个领域，若它们的输入\输出空间或概率分布中至少有一个不同，则两个领域就被认为是不同的

▶ 迁移学习

- ▶ 指两个不同领域的知识迁移过程，利用源领域Source domain D_S 学习到的知识来帮助目标领域Target domain D_T 上的学习任务
- ▶ 源域的训练样本数量一般远大于目标领域

表 10.1 迁移学习和标准机器学习的比较

学习类型	样本空间	概率分布
标准机器学习	$\mathcal{X}_S = \mathcal{X}_T, \mathcal{Y}_S = \mathcal{Y}_T$	$p_S(x, y) = p_T(x, y)$
迁移学习	$\mathcal{X}_S \neq \mathcal{X}_T$ 或 $\mathcal{Y}_S \neq \mathcal{Y}_T$ 或 $p_S(x, y) \neq p_T(x, y)$	

迁移学习 (Transfer Learning)

► 迁移学习根据不同的迁移方式分为两个类型：

✓ **归纳迁移学习** Inductive Transfer Learning

- ① 对应归纳学习 Inductive learning，即希望在训练数据集上学习到使得期望风险（即真实数据分布上的错误率）最小的模型
- ② 归纳迁移学习是指在源领域和任务上学习出一般的规律，然后将这个规律迁移到目标领域和任务上（**模型迁移**）

✓ **转导迁移学习** Transductive Transfer Learning

- ① 对应转导学习 Transdactive learning，目标是学习一种在给定测试集上错误率最小的模型，在训练阶段可以利用测试集的信息
- ② 转导迁移学习是一种从样本到样本的迁移，直接利用源领域和目标领域的样本进行迁移学习（**样本迁移**）

迁移学习 (Transfer Learning)

► 归纳迁移学习

► 将预训练模型迁移到目标任务上通常比从零开始学习好:

- ✓1) 初始模型的性能一般比随机初始化的模型要好;
- ✓2) 训练时模型的学习速度比从零开始学习要快, 收敛性更好;
- ✓3) 模型的最终性能更好, 具有更好的泛化性

► 和多任务学习的区别:

- ✓1) 多任务学习是同时学习多个不同任务, 而归纳迁移学习通常分为两个阶段, 即源任务上的学习阶段和目标任务上的迁移学习阶段;
- ✓2) 归纳迁移学习是单向的知识迁移, 希望提高模型在目标任务上的性能, 而多任务学习是希望提高所有任务的性能。

迁移学习 (Transfer Learning)

▶ 转导迁移学习

- ▶ **转导迁移学习** 是一种从样本到样本的迁移，直接利用源领域和目标领域的样本进行迁移学习[Arnold 等人, 2007]。
- ▶ 转导迁移学习可以看作一种特殊的转导学习 (Transductive Learning) [Joachims, 1999]。
- ▶ 转导迁移学习通常假设源领域有大量的标注数据，而目标领域没有（或只有少量）标注数据，但是有大量的无标注数据。目标领域的数据在训练阶段是可见的。

终身学习 Lifelong learning

- ▶ 虽然深度学习在很多任务上取得了成功，但是其前提是训练数据和测试数据的分布要相同，一旦训练结束模型就保持固定，不再进行迭代更新。并且，要想一个模型同时在很多不同任务上都取得成功依然是一件十分困难的事情。
- ▶ 人类的学习是一直持续的，人脑可以通过记忆不断地累积学习到的知识，这些知识累积可以在不同的任务中持续进行。
- ▶ 在大脑的海马系统上，新的知识在以往知识的基础上被快速建立起来；之后经过长时间的处理，在大脑皮质区形成较难遗忘的长时记忆。由于不断的知识累积，人脑在学习新的任务时一般不需要太多的标注数据

终身学习 (Lifelong Learning)

▶ 终身学习

- ▶ 也叫持续学习 (continuous learning)
- ▶ 根据历史任务中学到的经验和知识，来帮助学习不断出现的新任务，并且这些经验和知识是持续累积的，不会因为新的任务而忘记旧的知识
- ▶ 假设一个终身学习算法已经在历史任务 $T_1, T_2, \dots, T_m$ 上学习到一个模型，当出现一个新的任务 T_{m+1} 时，这个算法可以根据过去在 m 个任务上学习的知识来帮助学习第 $m+1$ 个任务，同时累积所有的 $m+1$ 个任务上的知识。

终身学习 (Lifelong Learning)

▶ 终身学习

▶ 和迁移学习/多任务学习的区别：

- ✓ 知识累积：归纳迁移学习的目标是优化目标任务的性能，不关心知识的累积。而终身学习的目标是持续的学习和知识累积。
- ✓ 学习方式：终身学习并不在所有任务上同时学习。多任务学习是在使用所有任务的数据进行联合学习，并不是持续地一个一个的学习。

▶ 关键问题：在于如何避免灾难性遗忘

- ✓ 即学习新任务的同时不忘记先前学会的历史任务，比如在神经网络模型中，一些参数对任务TA 非常重要，如果在学习任务TB 时被改变了，就可能给任务TA造成不好的影响。
- ✓ 方法：弹性权重巩固

元学习

- ▶ 虽然根据没有免费午餐定理，没有一种通用的学习算法可以在所有任务上都有效。因此，当使用机器学习算法实现某个任务时，我们通常需要“就事论事”，根据任务的特点来选择合适的模型、损失函数、优化算法以及超参数。
- ▶ 那么，我们是否可以有一套自动方法，**根据不同任务来动态地选择合适的模型或动态地调整超参数呢？**事实上，人脑中的学习机制就具备这种能力。
- ▶ 在面对不同的任务时，人脑的学习机制并不相同。即使面对一个新的任务，人们往往也可以很快找到其学习方式。这种可以动态调整学习方式的能力，称为**元学习**（Meta-Learning），也称为**学习的学习**（Learning to Learn）[Thrun 等人，2012]。

元学习 (Meta Learning)

▶ 元学习

- ▶ 其目的是从已有任务中学习一种学习方法或元知识，可以加速新任务的学习，侧重从多种不同任务中归纳出一种学习方法
- ▶ 这里介绍两种典型的元学习方法：基于优化器的元学习和模型无关的元学习。

谢 谢!